

Where the Seconds Go:

A Speed Limit and a Measured Accounting for Mixture-of-Experts Decode on One Consumer GPU

Harshith Kantamneni
Independent Researcher

Abstract

A Mixture-of-Experts model too large for one consumer GPU keeps most experts in host memory. How fast could it decode, and where does the rest of the time go? We bound the time per token by the fewest expert reads any exact-routing policy can make, Belady’s optimum with bypass on the model’s own routing, over the machine’s host and GPU rates. On RTX 5090 hosts our llama.cpp expert cache reaches 25–43% of this limit, and published systems in its class a median of 13.6% of theirs. Oracles of future routing inside the engine then measure where the gap goes. On three hosts, foresight spent as the optimum spends it, admitting only what it admits and reading each admitted expert once, is worth 16–51% at every budget, and 25–81% with the overlap it allows; spent as caches usually spend it, by Belady prefetch or by copying what the CPU has just served, it reads 1.6–2.5 $\times$ the optimum’s bytes, loses at the lowest budgets, and trails the optimum’s way at every host-bound budget. Without foresight, a learned admission order fitted on other text reads 6–13% fewer experts but runs 0.90–1.05 $\times$ the deployed cache at the host-bound budgets; nine models’ routing traces give the horizon a forecaster must reach.

1 Introduction

Open-weight Mixture-of-Experts models now carry 30–120 B parameters and activate a few billion per token; a desktop GPU has 24–32 GB. Running such a model at batch 1 therefore means keeping most experts in host DRAM, and each decoded token reads its routed experts from one of three places: GPU memory, host memory by the CPU that runs the expert, or host memory across PCIe into a GPU slot [19, 15]. A line of systems builds on this: GPU expert caches with demand fetch [10, 48, 39], CPU execution of misses [22, 7, 58, 17, 57, 60], per-miss choices between the two [51], and expert prediction [18, 59, 46]. Each reports a speed-up over a baseline of its choosing. None states how fast its machine *could* decode the model within the same GPU memory, or what the rest of the time is made of.

Thesis. Where host memory binds, the time of offloaded decode is its host bytes per token over the host’s read rate. Belady’s optimum with bypass (MIN) sets the fewest such bytes; the best online policy reads 35–110% more on the routing traces, and our deployed cache, which reads its admissions twice, 1.66–2.27 $\times$ as many in the engine. Knowing future routing recovers that excess at every budget and on every host when it is spent as MIN spends it: admit what MIN admits, and read each admitted expert from host memory once. Spent the usual ways, it pays at some budgets and loses at others. We establish this in three steps. A *speed limit* in seconds (Section 3) bounds every exact-routing system with C expert slots per layer on a given machine. *Oracles* of future routing built into a real engine (Section 4) measure, on three hosts, two of them in randomised order, what foresight is worth under each way of spending it, separating the bytes it saves, the overlap it allows, the experts it admits that MIN would bypass, and the second read that copying a CPU-served miss costs. *Routing traces* of nine models (Section 5) say what would supply that foresight: no online policy does, batched speculative verification does not, and a forecaster must see $W_{50} \approx 0.59(C/k)^{1.33}$ tokens ahead to recover half of it.

The method is Hoeffler and Belli’s [16]: bounds beside measurements, paired statistics, and every prediction committed to a public repository before the machine it concerns was started (Appendix B scores all 565 clauses). We work on rented RTX 5090, 4090 and 3090 hosts with desktop CPUs, with gpt-oss-120b [33] and Qwen3-30B-A3B [50]. One example runs through the paper: gpt-oss-120b with 32 of each layer’s 128 experts resident on an RTX 5090, a quarter of its expert bytes. Table 1 defines the terms.

Contributions.

- **A speed limit in seconds** (Section 3) for exact-routing systems with C slots per layer, from the exact MIN on the model’s routing and the machine’s measured ceilings, varied four ways, stated for pooled caches, and placed beside our cache, FreeToken, llama.cpp and 52 published measurements.
- **A measured accounting of the gap** (Section 4): a factorial of oracle policies in the engine on 3 hosts,

Table 1: Terms used throughout.

cell	a model at a budget: the share of each layer’s experts resident on the GPU
E, k, C, L	experts per layer, experts per token, GPU slots per layer, MoE layers
miss	a requested expert not resident: read from host memory by the CPU or by a copy
fetch	copy a miss into a slot and run it there in the same step (one read)
admission	making an expert resident; <i>serve-then-copy</i> : the CPU serves the miss, then a background copy reads it again
prefetch, lead d	copy an expert into a slot at least d steps before its first use (one read)
paced	copies in 16 MB pieces, one in flight
MIN, R^*	Belady’s optimum with bypass on the routing trace; its host reads per token, the fewest of any policy
online / oracle	a policy that sees only past routing / one that reads future routing from a recorded trace
$B_c, B_p, B_{cp}, B_{\text{host}}$	probed host read rates: CPU, link, both, highest by any method
limit	Eq. (2): the speed bound of a machine for a cell
host-/GPU-bound	the limit binds on host reads / on GPU work
$v(F), v(O, F)$	the modelled time with foresight alone / with foresight and overlap (Appendix E)
W_{50}	tokens of perfect foresight that close half of an online policy’s excess reads
hosts B, S, A, O1–O5	rented RTX 5090 machines (Section 2)

which measures the terms a model of the gap prices and finds the model wrong about their sizes: at the host-bound cells the bytes foresight saves and the overlap it allows are together 38–70% of the gap (26–60% and 7–23% taken bytes first), and the rest, which no oracle touches, 30–62%.

- **What would recover it** (Section 5): across nine models’ traces, which online policy earns its reads, how fast the hot set drifts, how far ahead foresight must see, and why speculative batching cannot supply it once rejected drafts are paid for; and a learned admission order, fitted on other text with features the engine has when it plans a step, which removes 17–30% of a single-read policy’s excess reads offline, and in the engine reads fewer experts but runs 0.90–1.05× the deployed policy, gaining only where host memory is weakest.

The instrument is our llama.cpp expert cache (Section 2); it runs 2.0–4.0× stock llama.cpp and leads FreeToken [51] at 11 of 12 cells on two RTX 5090 hosts. Running missed experts on the CPU [22, 7], caching hot experts with CPU-executed misses [58, 17, 26], splitting a layer’s misses between CPU and PCIe from a bandwidth model [51, 60], bytes over bandwidth as a bound [53, 54], and Belady with bypass as a cache-level oracle [56] are published; our cache uses the first three.

2 Setting and Instrument

Workload. Single-request decode. gpt-oss-120b (MXFP4 experts; 36 MoE layers of 128 experts, top-4, 13.25 MB per expert) and Qwen3-30B-A3B in BF16 (48 layers of 128 experts, top-8, 9.44 MB per expert). Prompts are the 30 AIME-25 problems in FreeToken’s own benchmark format; each system decodes 256 tokens per problem greedily, after one warm-up request, in one session in which the cache carries over between problems. One client times every system with FreeToken’s own metric, (tokens−1) over the time between the first and last token’s arrival.

Machines. RTX 5090 (32 GB, 1.79 TB/s datasheet) hosts with desktop CPUs, rented by the hour and named by a letter. Table 2 is measured on *host B* (Ryzen 9 9950X3D; its card’s memory clock is $1.21\times$ the common one, and its probe read device memory 3% above the datasheet rate) and *host S* (Ryzen 9 9950X at the common clock; its CPU reads host memory 22% slower, its card has a lower power limit and SM clock). The oracle measurements of Section 4 use further rentals, O1–O5. Every job first runs a probe that reads host memory from t CPU threads, over PCIe, and from both at once: B_c is the CPU rate at the helper thread count, B_p the PCIe rate, B_{cp} the combined rate, and B_{host} the highest rate the probe reached by any method. GPU clocks cannot be locked on these rentals (power limits 350–600 W).

The instrument. Our cache is a 3,500-line patch to llama.cpp. Each MoE layer has C GPU slots. Routing is decided on the GPU; resident experts run there, and each miss either runs on CPU helper threads, which the GPU signals through a mailbox in mapped memory, or is fetched: copied into a slot and run on the GPU in the same step. Admission follows a decayed-frequency score (half-life 16 tokens, margin $\kappa=1$; a llama.cpp pull request admits by a decayed heat in the same spirit [31]); an admitted miss is served by the CPU and then copied into its slot in the background, so it is read from host memory twice. The one per-machine decision is how many of a layer’s n misses to fetch. A model of time per token gives it. With X_c the bytes the CPU reads per token and X_p the bytes the link reads,

$$T = G + \max\left(\frac{X_c}{B_c}, \frac{X_p}{B_p}, \frac{X_c + X_p}{B_{cp}}\right), \quad (1)$$

with one constant G per engine and model, fitted once on one host and frozen; its per-layer form picks, for each n , the number $f(n)$ of misses to fetch (Appendix C). In the running example on host B the table copies one of two misses and runs a single miss on the CPU, hidden behind the GPU’s resident experts. Written down on each machine from its probe before any model ran, Eq. (1) predicted our cache on gpt-oss within a median 3.4% per measurement on 7 RTX 5090 hosts (90th percentile 12%; Table 7). A one-path bytes-over-bandwidth model with

Table 2: Decode speed at equal GPU expert memory on two RTX 5090 hosts. 30 AIME-25 problems, first 256 decode tokens, greedy, session; our cache uses the FETCH split computed from each machine’s bandwidth probe; FreeToken uses its faster backend per budget, picked on host B on a separate launch and carried over to the second host. Ratios are of mean speeds, paired by problem, with 95% bootstrap intervals. *Speed limit*: the ceiling of Section 3 for an exact-routing system with the same slots per layer on that machine (exact optimum, that host’s highest probed rate, datasheet GPU rate, which host B’s card exceeds by 3%; Table 3 tightens it). FreeToken keeps one pooled cache and llama.cpp pins whole layers: against the pooled bound their designs allow they stand at 18–40% and 8–20%. With every weight in the VRAM of an RTX PRO 6000 (the RTX 5090’s datasheet bandwidth, 96 GB), stock llama.cpp decodes gpt-oss at 255 and Qwen3 at 178 tok/s. [†]Launch 1 for both systems: FreeToken’s backend was picked on the launch it is scored on, which favours it, and the row has no confirmation launch.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Speed limit	Ours, % of limit
Host B: Ryzen 9 9950X3D, card at 17001 MHz memory clock, CPU 72 GB/s, link 53, together 78								
gpt-oss-120b	11%	34.9	54.0 (hybrid)	69.9	1.294 [1.278, 1.312]	2.00×	172	41%
gpt-oss-120b	25%	40.0	85.6 (hybrid)	109.2	1.275 [1.254, 1.295]	2.73×	430	25%
gpt-oss-120b	40%	47.7	132.2 (offload)	152.5	1.154 [1.134, 1.172]	3.20×	519	29%
Qwen3-30B-A3B	12.5%	19.5	38.7 (hybrid)	40.0	1.032 [1.022, 1.042]	2.05×	96	42%
Qwen3-30B-A3B	25%	22.1	54.7 (hybrid)	63.1	1.153 [1.134, 1.173] [†]	2.86×	214	30%
Qwen3-30B-A3B	43.75%	28.4	103.1 (offload)	108.2	1.049 [1.036, 1.064]	3.81×	308	35%
Host S: Ryzen 9 9950X at the stock clock, card at 14001 MHz, CPU 56 GB/s, link 53, together 66; FreeToken’s backend carried over from host B								
gpt-oss-120b	11%	28.9	48.0 (hybrid)	57.9	1.207 [1.195, 1.219]	2.00×	140	41%
gpt-oss-120b	25%	33.6	78.7 (hybrid)	94.1	1.196 [1.163, 1.224]	2.80×	351	27%
gpt-oss-120b	40%	40.1	122.2 (offload)	133.6	1.093 [1.077, 1.111]	3.34×	514	26%
Qwen3-30B-A3B	12.5%	16.1	32.7 (hybrid)	33.5	1.027 [1.018, 1.036]	2.08×	78	43%
Qwen3-30B-A3B	25%	18.5	51.4 (hybrid)	53.9	1.048 [1.037, 1.059]	2.91×	174	31%
Qwen3-30B-A3B	43.75%	23.8	98.3 (offload)	95.8	0.974 [0.962, 0.987]	4.02×	305	31%

the same G has the same median but a 90th percentile of 31%: the maximum over paths matters where both paths carry many bytes. With its constants frozen it scores 30 later measurements on 4 more hosts at a median 5.6%, within -6 to +4% on gpt-oss and over-predicting Qwen3’s speed by +5 to +21%, whose G was borrowed from llama.cpp; on those later measurements the one-path model does as well (median 6.1%, 90th percentile 11 against 16%). It is a calibrated model of engines that read host memory in sequence with their GPU work, and Section 4 shows where it stops: it prices no overlap and no per-layer fetch latency.

Protocol. We match the number of experts on the GPU across systems (within 2.5%; llama.cpp keeps whole layers, `--n-cpu-moe n`, and FreeToken takes a cache rate and preallocates most of the card). Ratios are of mean speeds, paired by problem, with 95% percentile intervals from 10,000 bootstrap resamples; except in Section 4, they cover problem sampling within one launch on one machine, not rentals (two rentals of one CPU model differed by 19–22% in speed). A first launch picks each system’s faster variant per budget and a second launch, which the comparison uses, runs only those. Every job’s predictions are in its script’s header, committed before the machine starts. A model’s error is its prediction over the measurement, minus one, of the quantity named, speed or time per token; a model that over-predicts time under-predicts speed.

The instrument against its peers. Table 2 compares three systems at six cells on two RTX 5090 hosts. On host

B ours leads FreeToken at every budget, by 15–29% on gpt-oss and 3–15% on Qwen3; on host S, with a slower CPU, it leads at five of six cells and FreeToken leads at Qwen3 43.75%. Tuned per cell on a third host (a Ryzen 9 9950X) over five settings (its fetch cap, its thread count and its backend), FreeToken’s best runs at most 1.04× its Table 2 setting, and ours runs 0.98–1.17× it, ahead at 5 of six cells (job 098; Appendix G). FreeToken reserves 28.4–30.1 GiB of VRAM at every budget, ours uses 9.5–27.3 GiB. With every expert on the CPU, stock llama.cpp, whose kernels our helpers use, runs 1.13× (gpt-oss) and 0.99× (Qwen3) FreeToken’s CPU backend, so CPU kernels explain none of the Qwen3 lead and at most part of the gpt-oss one. Most of the lead is admission rationing: with LRU in place of decayed frequency the cache runs 6–21% slower and trails FreeToken at four cells (Appendix G). Ours runs 1.6–2.6× llama.cpp on an RTX 4090 and an RTX 3090, ties FreeToken on its own headline model within 3%, and ties llama.cpp on Mixtral-8x7B, where 2 of 8 experts resident leave little locality to earn (Appendix D). Its outputs diverge from an all-VRAM reference as much as stock llama.cpp’s own CPU placement does (mean KL 0.0019 and 0.0005 nats; Appendix G).

3 The Speed Limit

Bound. Consider any system that executes the model’s exact routing with at most C experts of each layer resident on the GPU. Per token, each of the Lk routed experts is either served by a resident copy, which the GPU reads from VRAM, or not, in which case its S bytes are read

Table 3: The speed limit under each tightening, tokens per second on host B, and our cache as a percentage of the limit as published (exact optimum, highest probed host rate, datasheet GPU rate) and with every tightening at once. *no-evict*: Belady’s MIN with bypass that keeps a served expert for the step; *exact*: the same, free to evict an expert after serving it; *pool*: one pool of *LC* slots shared across layers; *median*: B_{host} the median of six concurrent probe samples instead of the highest; *GPU*: the GPU term from the measured all-in-VRAM time instead of the datasheet rate; *layers*: the per-layer sum, which no system without cross-layer prefetch can beat; *all*: exact, pool, median and GPU together (a warm start, not included, changes the limit by at most 1.4%).

Model	Experts on GPU	no-evict	exact	pool	median	GPU	layers	all	Ours, % of exact	all
gpt-oss-120b	11%	169	172	182	159	172	163	168	41	41
gpt-oss-120b	25%	427	430	474	397	279	365	278	25	39
gpt-oss-120b	40%	519	519	519	517	279	500	278	29	55
Qwen3-30B-A3B	12.5%	92	96	101	89	96	93	93	42	43
Qwen3-30B-A3B	25%	209	214	234	197	192	193	191	30	33
Qwen3-30B-A3B	43.75%	308	308	308	307	192	298	191	35	57

from host memory: by the CPU that runs it, or by the copy that brings it into a slot. Let R^* be the reads per token of MIN, Belady’s optimum with bypass [3, 12, 55], applied per layer to the routing trace: the fewest reads of any policy, even one that knows the future. If c experts per token run on the CPU, host memory supplies at least $\max(R^*, c)S$ bytes and the GPU reads the dense bytes D plus $(Lk - c)S$. With everything overlapped,

$$\bar{T} \geq \min_{0 \leq c \leq Lk} \max \left(\frac{D + (Lk - c)S}{B_{\text{gpu}}}, \frac{\max(R^*, c)S}{B_{\text{host}}} \right), \quad (2)$$

with B_{gpu} the GPU’s datasheet rate. A cell is *host-bound* when the minimum is at $c^*=0$, and *GPU-bound* otherwise: the GPU binds with every expert it can take. On host B gpt-oss at 11 and 25% and Qwen3 at 12.5 and 25% are host-bound. The bound’s class is exact routing, whole experts of unmodified bytes, at most C of each layer resident, and one token per forward pass. A system free to pool its *LC* slots across layers has a bound 0–10% higher; FreeToken keeps one LRU over all layers and llama.cpp pins whole layers, so we score both against the pooled bound. The bound assumes an expert run on the CPU is read from DRAM (the optimum’s shortest re-read distance is 0.5 GB of host traffic), that computation runs at memory speed, and that the two read paths share B_{host} freely; it drops all latencies. B_{host} is the highest rate our probe reached, not a hardware peak, so a faster reader could exceed it. The traces are each model’s own greedy text on the 30 prompts, with routing recorded teacher-forced; replayed through a simulator of our deployed policy they reproduce the engine’s hit rates within 5.3 points.

Varying it. Table 3 changes one assumption at a time. Pooled slots need 4.5–18.6% fewer reads and raise the

host-bound limits by 5–10%. Starting the optimum warm changes it by at most 1.4%; the median probe sample instead of the highest lowers the host-bound limits by 8%. The GPU term moves most: at batch 1 the all-in-VRAM runs reach only 52% (gpt-oss) and 61% (Qwen3) of the datasheet rate, and with the measured rate the limits at 25% and above fall to 279 and 192 tok/s, 8–9% above the all-in-VRAM speed: the CPU’s share. In the running example the limit on host B is 430 tok/s (279 with the measured GPU rate) and the cache runs 109 tok/s.

Applying it. Against the limit our cache stands at 25–43% on the two hosts of Table 2, and at 33–57% with every variation of Table 3 at once; FreeToken and llama.cpp stand at 18–40% and 8–20% of the pooled bound their designs allow. With every expert in its slots the cache matches stock llama.cpp ($0.997\times$ and $1.047\times$): it costs nothing when the model fits. We also scored 52 published batch-1 measurements from 13 sources against a bound with the same ingredients at each paper’s datasheet rates, in a per-layer form that charges dense work in sequence (`scripts/audit.py`; every row in Appendix H). The 20 that are inside the bound’s class and have a routing trace of their model reach a median 13.6% of it (quartiles 8.2–23.1%). The ceilings are datasheet ones and 21 rows’ capacities are imputed upper bounds, so these are lower than a probed limit would give; even so, three quarters of the in-class published systems stand below a quarter of a latency-free ceiling, and the next section asks what the missing part is made of.

4 Where the Seconds Go

Between a cell’s measured time and its limit, Eqs. (1) and (2) price five fixes; their Shapley values (Appendix E) name missing foresight across tokens the largest cost at host B’s four host-bound cells, 37–53% of the gap, and host reads serialised with GPU work the largest at its two GPU-bound cells. That is a model, and the states it values in between are not realised by any online policy. This section realises them: oracles of future routing built into the engine, on three hosts, measure what foresight is worth under each way of spending it, and find the model right about which terms matter and wrong about their sizes.

The experiment. Each oracle reads the routing of the rest of the sequence from a trace recorded on the same machine over the same teacher-forced text, and the engine runs it beside the online policy at every cell (Fig. 1). On O4 (Ryzen 9 9950X) and O5 (Ryzen 9 9950X3D) the ten configurations of a cell ran in an order shuffled per cell, each in a fresh context, with the predictions for both hosts committed before either started (Appendix B); a second launch on O4’s machine (job 097), in another random

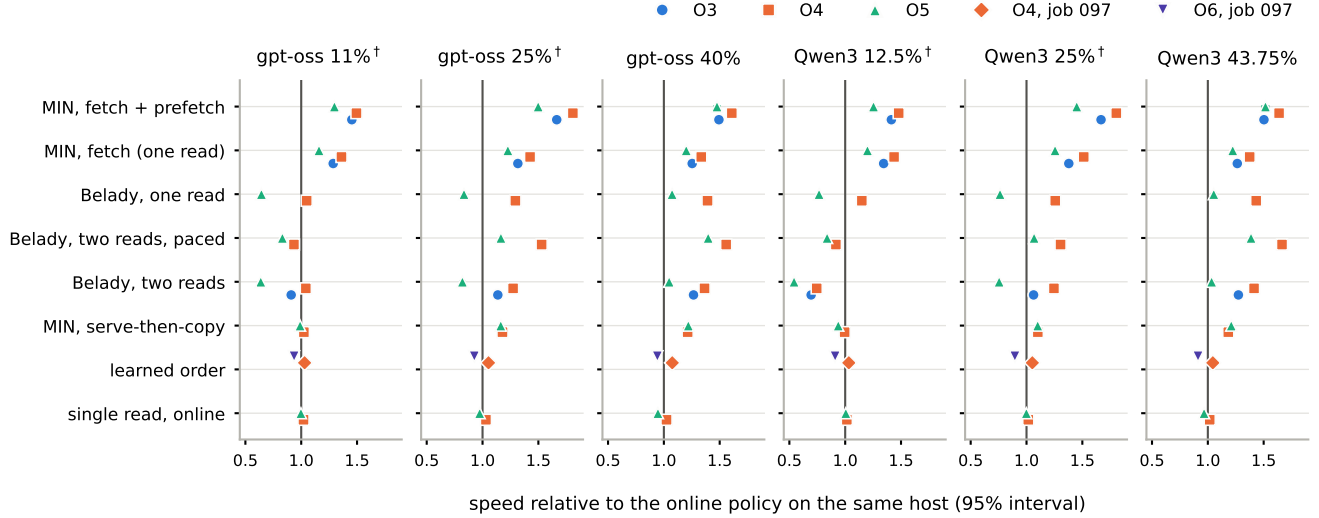


Figure 1: Foresight in the engine: the speed of each state relative to the online policy on the same host, with 95% paired intervals (mostly narrower than the markers), at the six cells of Table 2, on hosts O3 (job 095, fixed order), O4 and O5 (job 096, shuffled order, cold cache for each configuration). Rows, bottom to top: the online policy with each of its admissions read once (no foresight); the same with the learned admission order of Section 5 (job 097, on O4’s machine and host O6, a Ryzen 9 9950X3D); MIN with bypass with its admissions served by the CPU and then copied (two reads); Belady prefetch of the coming steps, which admits what MIN bypasses, through that same path (two reads), the same on the paced copy path, and with each admission read once; MIN with bypass with each admission fetched into its slot in the step that needs it (one read); the same with a paced single-read prefetch three steps ahead. O5’s link reads 27 GB/s against 43 and 45 on O3 and O4. †host-bound. Milliseconds and reads per state: Tables 9 and 10.

order, reproduced the online, single-read, fetch and paced-prefetch ratios within 0.024. The configurations cross two choices a cache with foresight makes. What to admit: what MIN with bypass admits, or every expert the coming steps will request, which is what a Belady prefetch does, admitting experts MIN would bypass. How to read an admitted expert: once, by fetching it into its slot in the step that needs it or by copying it ahead of its first use; or twice, by serving it on the CPU and then copying it, the way caches that execute misses on the CPU admit, ours included. One more configuration reads the online policy’s own admissions once, with no foresight.

Spent as the optimum spends it, foresight pays.

MIN with bypass with each admission fetched once runs $1.36\text{--}1.51\times$ the online policy at the four host-bound cells on O4 and $1.16\text{--}1.26\times$ on O5, and gains 16–51% at every cell on the three hosts. It reads $1.04\text{--}1.14\times R^*$, within 3% of what the simulator predicts for it. With a paced single-read prefetch three steps ahead the gain is 25–81% ($1.48\text{--}1.81\times$ on O4 and $1.25\text{--}1.50\times$ on O5 at the host-bound cells), bringing the engine to 46–73% of the limit there against the online policy’s 31–49%. Every interval is within ± 0.037 . On both hosts the paced prefetching state beats fetch and fetch beats the online policy with single reads at every host-bound cell; the sizes depend on the host. Fetch’s ratio differs between O4 and O5 by 0.13–0.26 (we had predicted at most 0.15), consistent with

the in-step fetch waiting on O5’s slower link, though the two hosts also differ in their CPU.

Spent the usual ways, it pays less, and at the lowest budgets nothing.

Two habits waste it. The first is over-admission: Belady prefetch admits what MIN bypasses and reads $1.6\text{--}2.3\times R^*$ even with each admission read once. The second is serve-then-copy: MIN with bypass whose admissions the CPU serves and then copies reads $1.7\text{--}2.1\times R^*$. Their value depends on the budget and the host. At the two lowest budgets the best of the usual ways (Belady prefetch read once or twice, paced or not, and serve-then-copy MIN) runs $0.94\text{--}1.15\times$ the online policy on O4 and O5, against $1.16\text{--}1.44\times$ for MIN fetched once; on O5 the best of them runs $0.94\text{--}1.16\times$ at the four host-bound cells. At the two 25% cells on O4 they gain more ($1.30\text{--}1.53\times$, the paced Belady prefetch once beating MIN fetched in the step), as the paced Belady prefetch did on O1 and O2 (Appendix F). At every host-bound cell on O3–O5, MIN with its paced single-read prefetch beats the best of the usual ways by $1.18\text{--}2.03\times$. Reading Belady’s admissions a second time costs little at gpt-oss (the two forms are within $1.01\text{--}1.03\times$ of each other) and a factor of $1.41\text{--}1.54$ at Qwen3 12.5%, where admissions are most frequent. The online policy’s own second read, by contrast, costs it little: at the host-bound cells MIN admits 2–11 times as many experts per token as the online policy, whose 3.5–6.8 admissions per token the engine

rations, and reading those once removes 4–15% of its host reads but moves its speed by only $0.97\text{--}1.03\times$, consistent with their background copies overlapping later steps. The online policy’s time goes to not knowing the future, and knowing it pays at every budget only when spent on what MIN admits, each admission read once.

The accounting, measured. With the limit, the states online, fetch and the faster prefetching state divide the gap at the host-bound cells of O3–O5 into the bytes foresight saves, 26–60% of it; the overlap foresight allows, 7–23%; and the rest, which no oracle touches, 30–62% (Table 10). On O4 and O5, where the online policy also ran with single reads, reading its admissions once accounts for -4 to 4% of the gap and foresight for 26–57%. What the oracles recover, bytes and overlap together, is most of the gap on O3 and O4 (57–61% and 64–70%); on O5, with its slow link, the rest is the larger share (52–62%). The split between bytes and overlap depends on the order in which they are taken: taken overlap first, with the Belady prefetch standing for overlap at about the online policy’s bytes, the overlap share ranges from -161 to 32% (it is negative where that prefetch loses) and the bytes share from 34 to 199%; only their sum, 38–70%, and the rest are free of the order. Against the first order, the model over-prices the bytes $1.1\text{--}2.1\times$ and the overlap $1.1\text{--}6.4\times$ and under-prices the rest $2.1\text{--}8.4\times$. We did not separate the rest, but three candidates are visible. The prefetch is imprecise: on O3 its reads are $1.18\text{--}1.58\times R^*$ across the six cells, the cost of choosing victims steps early and of the misses that fall inside the lead. The overlap is partial: the law, which charges every host read in sequence with the GPU’s work, over-predicts the prefetching state’s time by +1 to +38% at the host-bound cells, so some reads overlap and some do not. And the GPU runs at 52–61% of the datasheet rate the limit counts, a large part of what remains at the GPU-bound cells: on Qwen3 at 43.75% Nsight Systems splits a token of our cache (FreeToken) into 4.93 (5.20) ms of expert copies over PCIe, 2.29 (2.21) of resident-expert products, 1.65 (1.34) of attention projections, 0.49 (0.37) of attention, 0.38 of output head and 0.97 (0.40) of norms, small kernels and cache control, with neither system overlapping its copies with GPU compute. Two things the model prices as overlap are worth nothing here: turning off the within-layer overlap of a layer’s CPU misses with its GPU hits changes speed by under 1% at every cell, and predicting the next layer’s experts [18, 59] never lowers host bytes. Foresight across tokens is the lever; on O1, half of the Belady prefetch’s gain at gpt-oss 25% arrived between windows of 4 and 16 tokens (9.2 by log interpolation), the horizon Section 5 finds on the traces.

Table 4: Host reads per token relative to the exact optimum (Belady’s MIN with bypass over the whole future), median over nine models at three budgets, and the range over all 27 model–budget cells. Event-atomic replay of each model’s own sampled text; every online policy is scored on the last 90% of the trace. Mixtral (8 experts) uses $C = 2, 3$ and 4.

Policy	$C=E/8$	$E/4$	$3E/8$	range
LRU	1.44	1.77	2.03	1.37–2.26
LFU	1.44	1.93	2.40	1.37–6.15
Decayed frequency	1.45	1.70	1.94	1.35–2.10
with hysteresis κ	1.46	1.75	2.00	1.36–2.55
ARC	1.44	1.76	1.99	1.37–2.23
S3-FIFO	1.44	1.69	1.94	1.38–2.11
Static, profiled	1.92	2.74	3.48	1.43–13.24
Belady, $W=1$	1.18	1.45	1.69	1.07–2.06
Belady, $W=4$	1.01	1.13	1.31	1.00–1.95
Belady, $W=16$	1.00	1.00	1.01	1.00–1.66

5 What the Traces Say

The accounting names foresight across tokens as the cost the oracles can remove where the host binds. What would recover it, and what would not? We replay the models’ own sampled text for 9 MoE models [32, 50, 33] (9 traces of 33,000–152,000 tokens, the cache carried across conversations) through an event-atomic simulator: each token requests its k experts per layer as one set, and no expert of the current token is evicted to serve it, the accounting that cache-level studies of this problem call for [56].

Policies. Table 4 scores LRU, LFU, decayed frequency, ARC, S3-FIFO and a profiled static cache against the exact optimum and against Belady within a window of W future tokens, at three budgets per model. Three things hold across models. First, the online policies are close to one another and far from the optimum: the best of them reads 35–110% more than it, while decayed frequency is the best online policy in 12 of 27 cells and S3-FIFO in 10 (within 1% of the best in 20 and 18). Where the cache turns over every token ($C/k \leq 1.5$, 7 cells) the six online policies are within 4% of one another; from $C/k=2$ LRU reads 2–11% more than decayed frequency on eight of nine models, Qwen2-57B being the exception, and classic LFU fails at large budgets ($6.2\times$ the optimum). The literature’s disagreement on recency against frequency [38, 23, 49, 14] is consistent with this regime dependence. Second, a static cache profiled on other text reads $1.4\text{--}13.2\times$ the optimum, because the hot set moves: the C most requested experts of a layer change by 22–58% between consecutive windows of 100 tokens and 33–75% between windows of 1,000 (median 51%), so popularity follows the conversation; the skew is mild, the top quarter of experts taking 31–56% of requests. Third, four tokens of foresight beat every online policy in

every cell (median $1.13\times$ the optimum at $E/4$), so foresight is worth several times the choice of policy. Hysteresis as the trace study sets it ($\kappa=k/4$: 1 for gpt-oss, 2 for Qwen3; the engine runs $\kappa=1$) costs 1–8% reads over plain decayed frequency on eight models and up to 22% on Qwen3; it is tuned for time, where it rations copies, not for reads.

How far ahead. The optimum reads 26–56% fewer experts from host memory than decayed frequency with each admitted miss fetched once, the single-read form of our policy (median 41%; against the serve-then-copy form, which reads its admissions twice, the saving is larger). The lookahead needed scales with the cache’s turnover time C/k , slots per layer over experts per token: half of the gap closes at $W_{50} \approx 0.59 (C/k)^{1.33}$ tokens ($r=0.971$ in log-log over 26 points; the exponent’s 95% interval over a bootstrap of models is 1.09–1.47, its leave-one-model-out range 1.24–1.37, and without the 6 points whose W_{50} lies below one token it is 1.51), and 90% needs a median of $2.7 C/k$. Small caches ($C/k \leq 2$) recover half of the gap with one or two tokens of foresight; gpt-oss-120b at 25% ($C/k=8$), the running example, needs about 10 tokens, and at 50% about 34 (both measured on its trace; the fit gives 9 and 23). A prefetching system needs that horizon plus the lead of its copy path, two to five steps in ours. The law adds the condition: fewer reads are worth most where host memory is the binding term, and a system that streams every expert through a saturated bus each token, as on the devices of [54, 53], has no slots to fill and nothing to foresee.

Foresight from history. No forecaster here sees W_{50} tokens ahead, but part of the saving needs none. We trained a logistic model of whether an expert is requested within the next C/k tokens on features the engine has when it plans a step: decayed request counts at five half-lives, the time since the last request, whether the expert is requested now, and transition scores from the previous token’s experts at the same layer and, through an 8-bit table, at every layer. It is fitted on each model’s mixed-domain trace and replayed on the AIME routing the engine runs, so no test text is seen in training. Ordering admissions and victims by it, with each admitted expert read once, reads 6–12% fewer experts than decayed frequency with the same single-read admission at the four host-bound cells, closing 17–30% of that policy’s excess over MIN (Table 5; 14–26% without the cross-layer scores), and at the 3 cells with $C/k \geq 7$ it reads less than Belady with one token of perfect foresight. It still reads 1.43 – $1.69\times$ MIN at the host-bound cells. The replay, the model’s weights and a gate for measuring it in the engine were committed before any engine run of it (prereg/learned_gate_097.md). In the engine (job 097, predictions committed before launch) it reads 6–13% fewer experts than the online policy with single reads at the host-bound cells, on both hosts, but its speed depends on the host. On O4’s machine (host memory 50 GB/s at best) it runs 1.01 – $1.03\times$ the single-

Table 5: A learned admission order, offline: host reads per token on the AIME-25 routing the engine runs, of the decayed frequency with each admission read once (single read; within 10% of the engine’s fetch-on-admit reads on hosts O4 and O5), the learned order with single-read admission (trained on the model’s mixed-domain trace; 8-bit cross-layer table, as in the engine), Belady within one and four tokens of perfect foresight, and MIN with bypass in its no-evict form (Table 3). *closed*: the share of the single-read policy’s excess over MIN that the learned order removes. [†]host-bound.

cell	single read	learned	$W=1$	$W=4$	MIN	closed
gpt-oss 11% [†]	59.4	56.0	54.4	47.3	39.0	17%
gpt-oss 25% [†]	28.3	26.1	27.3	25.1	15.5	17%
gpt-oss 40%	14.3	12.6	14.0	13.2	6.8	23%
Qwen3 12.5% [†]	166.2	146.3	134.1	109.1	100.4	30%
Qwen3 25% [†]	84.2	74.8	74.1	62.8	44.3	24%
Qwen3 43.75%	28.8	26.3	27.7	25.3	13.8	17%

read online policy and 1.03 – $1.05\times$ the deployed one; on host O6, a Ryzen 9 9950X3D whose host memory reads 69 GB/s, it runs 0.90 – $0.94\times$ and 0.90 – $0.94\times$, slower. The reads it saves are CPU reads; it buys them with 1.11 – $1.60\times$ as many fetches in the step, each a copy its layer waits on, and with 237 – $585\mu\text{s}$ of host time per token for its logits, a trade that paid on the host with the weaker host memory and lost on the stronger one. A realisable order from routing history is worth at most a few percent here, and nothing on a stronger host; the value Section 4 measures needs foresight.

Speculation is not foresight. Speculative decoding verifies K draft tokens in one forward pass, so a layer’s experts for K positions are requested together and a missed expert is read once for all of them, which several systems propose as the source of foresight for a cache [8, 27, 35]. On the same traces, with K positions per step and drafts accepted at rate α until the first rejection, batched verification saves an online cache a median 15–32% of its reads across models at $K=8$ when rejected drafts cost nothing (48% on the model that gains most), the equivalent of a window of two to four tokens; once the rejected drafts’ experts are counted, the saving is 8–12% at $\alpha=0.9$ and becomes a loss of 18–20% at $\alpha=0.8$, because the union of K positions’ expert sets is 20–79% of Kk across models, budgets and variants (Appendix B lists the variants). The optimum gains little from batching (–7 to 15%). Foresight must come from knowing future routing, as a router decoupled from the backbone [4] or a forecaster [46] would provide, and W_{50} says how far ahead.

6 Related Work

Offloading systems. Demand-paging systems cache experts per layer and copy misses over PCIe [10, 48, 39, 42, 36]. CPU-execution systems run non-resident experts

on the host [22, 7, 58, 57, 45]; a set-associative GPU cache with CPU-executed misses [17], a llama.cpp fork [25, 26] and two llama.cpp pull requests [31, 9] are the closest designs to ours, and FreeToken [51] and DALI [60] split each layer’s misses between CPU and PCIe, FreeToken from a closed-form bandwidth model. Pipelined Sharding [43] plans sub-layer placement under a VRAM budget. Predictors guess upcoming experts from the previous layer’s state [18, 59], several steps ahead with a learned forecaster [46, 49], or a batch’s routes across layers [13]; ReadME [4] makes routing known in advance by decoupling the router and then caches in Belady’s manner. Speculative decoding has been combined with expert offloading as a source of batching [8, 27, 35]. Batched-throughput systems [37, 6, 52] and SSD tiers [29, 30] are outside our single-request, host-DRAM scope.

Models, bounds and audits. FreeToken’s split and our Eq. (3) are bandwidth models of one layer; Eq. (1) is a whole-token model tested blind across machines and against alternatives. Max-of-paths cost models go back to FlexGen [37] and MoE-Lightning [6]; WiSP [54] bounds decode by PCIe turnover, Budgeting Bytes [53] by bytes over bandwidth, and MoE-CAP [21] measures bandwidth utilisation against a single peak. Qiu et al. [34] predict llama.cpp throughput from GGUF metadata for models held entirely in GPU or unified memory; Suram [41] matches one offloaded configuration to bytes per token over bandwidth; Si et al. [38] derive when a copy pays off and find recency competitive with frequency at large budgets. The roofline [47] is the single-memory ancestor of our bound. Zhang [56] proposes a trace-level evaluation contract for expert caching, decomposes the online-to-optimal gap in cache blocks in one fixed order, and audits the reporting of ten papers; SeqMoE [46] reports four systems against the all-in-VRAM speed. Trace studies of policies cover two or three models each [56, 38, 23, 14]. Shapley attribution of a performance gap has a precedent in ShapleyIQ [28]. The choice between prefetching what the optimum would bypass and admitting conservatively is the integrated prefetching-and-caching trade-off of Cao et al. [5], whose offline schedules are near-optimal for one and several disks [24, 2]; our oracles re-derive it in this setting. Learned replacement imitates Belady from past accesses in CPU caches [20] and content-delivery caches [40], and FlashMoE [23] learns a recency–frequency blend for an expert cache on SSD; Section 5 measures what such an order recovers of the optimum’s saving once each admitted expert is read once. Limited lookahead in paging goes back to Albers [1]; Hasslinger et al. [12] combine Belady within a window with LRU or LFU beyond it for web caches, the policy family we use to measure the horizon.

7 Limitations

- **Hardware.** Consumer cards (RTX 5090 for the headline comparisons, a 4090 and a 3090 at the cells that fit 24 GB; an RTX PRO 6000 for all-in-VRAM runs) and desktop Ryzen and Intel hosts, rented; GPU clocks could not be locked, power limits differed (350–600 W), and host B’s card reads device memory 3% above the datasheet rate, where the common-clock cards read below it; Table 2 therefore reports host S beside it, where our lead is 0.01–0.11 smaller and FreeToken leads at one cell. A 12-channel server CPU breaks the law’s bandwidth assumption (Appendix C). Intervals cover problem sampling on one machine, not machine-to-machine variance, which Fig. 3 shows across eight hosts; the oracle factorial ran on three hosts, one launch each; fetch’s gain differs between O4 and O5 by up to 0.26 and other states’ by up to 0.49.
- **Workload.** Single-request decode; AIME prompts, which were also used while developing the system; greedy decoding; prompts of 52–879 tokens (median 130 for Qwen3, 186 for gpt-oss). Long outputs are measured on held-out problems. Prompts and batches above one bypass the cache and run 79–83% and 77–79% of stock llama.cpp’s speed; speculative verification bypasses it and is not measured.
- **Competitor settings.** Table 2 runs FreeToken’s faster backend per budget with its own bandwidth calibration, default thread count and fetch cap; tuned over five settings on another host (job 098) it gains at most 1.04×. Its CPU backend failed on a hybrid-core host at its default thread count and was rerun on the performance cores. Each comparison uses one confirmation launch, except Qwen3 at 25%, which is from the selection launch for both systems.
- **The bound, the accounting and the audit.** Exact routing only: expert skipping, substitution or deferral change the model and fall outside it; a per-layer budget, which a pooled cache can beat by 5–10%. Two of the model’s five terms are definitions (Appendix E). The oracles read a recorded trace, not a predictor; they measure what this engine does with foresight, and their prefetch still reads 1.18–1.58× the optimum’s bytes. The published rows use datasheet ceilings and, for eleven of them, i.i.d. routing.
- **Constants and outputs.** G , g and L_c are fitted once and frozen; the per-layer model overstates differences between FETCH tables about 1.8×. Output parity is measured by teacher-forced KL divergence, agreement and loss against an all-VRAM reference, not by task accuracy.

8 Conclusion

On a consumer machine, the optimum’s host bytes over the probed host rate and the datasheet GPU rate give a speed limit in seconds; our cache reaches 25–43% of it, and published systems in its class a median of 13.6% of a datasheet version. A calibrated host-memory model, predicted blind, places an engine in the limit’s coordinates and picks the path for a miss on a given machine. Where the host binds, the piece of the gap that knowing future routing removes is recovered at every budget, on every host we measured, only when that knowledge is spent as the optimum spends it, admitting what MIN with bypass admits and reading each admitted expert once: so spent, it runs our cache 16–51% faster on the bytes alone and 25–81% faster with the overlap it allows. Spent as a Belady prefetch or through serve-then-copy admission, it reads 1.6–2.5 $\times$ the optimum’s bytes, gains at some budgets and loses at the lowest, and trails the optimum’s way at every host-bound cell. The online policy’s own second read is not where its time goes. What remains after foresight is spent well is 30–62% of the gap, which we did not separate: the prefetch’s excess reads, reads that do not overlap, and GPU kernels at half the datasheet rate. Nine models’ traces say that neither a hand-built online policy nor speculative batching buys that foresight, that a learned order fitted on other text removes 17–30% of a single-read policy’s excess reads, and that in the engine such an order is worth 0.90–1.05 $\times$, so the rest must come from knowing the routing a few to tens of tokens ahead; W_{50} says how many.

References

- [1] Susanne Albers. On the influence of lookahead in competitive paging algorithms. *Algorithmica*, 18(3):283–305, 1997.
- [2] Susanne Albers, Naveen Garg, and Stefano Leonardi. Minimizing stall time in single and parallel disk systems. *Journal of the ACM*, 47(6):969–986, 2000.
- [3] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [4] Ruisi Cai, Yeonju Ro, Geon-Woo Kim, Peihao Wang, Babak Ehteshami Bejnordi, Aditya Akella, and Zhangyang Wang. Read-ME: Refactorizing LLMs as router-decoupled mixture of experts with system co-design. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 116126–116148, 2024. arXiv:2410.19123.
- [5] Pei Cao, Edward W. Felten, Anna R. Karlin, and Kai Li. A study of integrated prefetching and caching strategies. In *Proceedings of ACM SIGMETRICS*, 1995.
- [6] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- [7] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025.
- [8] Liangkun Chen, Zijian Wen, Tian Wu, Xiaoxi Zhang, and Chuan Wu. SP-MoE: Speculative decoding and prefetching for accelerating MoE-based model inference. *arXiv preprint arXiv:2510.10302*, 2025.
- [9] csantiago78. llama.cpp pull request #27861: GPU-resident LRU cache for host-offloaded MoE expert weights. <https://github.com/ggml-org/llama.cpp/pull/27861>, 2026. Draft, accessed September 2026.
- [10] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [11] Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. *arXiv preprint arXiv:2502.12224*, 2025.
- [12] Gerhard Hasslinger, Juho Heikkinen, Konstantinos Ntougias, Frank Hasslinger, and Oliver Hohlfeld. Optimum caching versus LRU and LFU: Comparison and combined limited look-ahead strategies. In *Proceedings of the 16th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, pages 1–6, 2018.
- [13] Xin He, Shunkang Zhang, Kaijie Tang, Shaohuai Shi, Yuxin Wang, Zihao Zeng, Zhenheng Tang, Xiaowen Chu, Haiyan Yin, Ivor W. Tsang, and Yew Soon Ong. Expert-Flow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling. *arXiv preprint arXiv:2410.17954*, 2024. Accepted at DAC 2026 (per arXiv v2).
- [14] Duc Hoang, Ajay Jaiswal, Mohammad Samragh, and Minsik Cho. SpecMD: A comprehensive study on speculative expert prefetching. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03921.
- [15] Torsten Hoeffler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [16] Torsten Hoeffler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [17] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [18] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. arXiv:2308.12066.

- [19] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [20] Akanksha Jain and Calvin Lin. Back to the future: Leveraging Belady’s algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016.
- [21] Yinsicheng Jiang, Yao Fu, Yeqi Huang, et al. MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2412.07067.
- [22] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [23] Byeongju Kim, Jungwan Lee, Donghyeon Han, Hoi-Jun Yoo, and Sangyeob Kim. FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices. *arXiv preprint arXiv:2601.17063*, 2026.
- [24] Tracy Kimbrel and Anna R. Karlin. Near-optimal parallel prefetching and caching. *SIAM Journal on Computing*, 29(4):1051–1082, 2000.
- [25] leloch. MoE expert cache for llama.cpp: VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. <https://github.com/leloch/llama.cpp/tree/moe-cache-v2-pr>, 2026. Branch moe-cache-v2-pr, commit e3096b0 (6 August 2026). Proposed upstream as llama.cpp pull request #24524 and RFC discussion #24528; not merged. Accessed September 2026.
- [26] leloch. RFC: MoE expert cache, VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. llama.cpp discussion #24528, <https://github.com/ggml-org/llama.cpp/discussions/24528>, 2026. Posted 12 June 2026; accessed September 2026.
- [27] Shuhuai Li, Jianghao Lin, Dongdong Ge, and Yinyu Ye. MoE-SpAc: Efficient MoE inference based on speculative activation utility in heterogeneous edge scenarios. *arXiv preprint arXiv:2603.09983*, 2026.
- [28] Ye Li, Jian Tan, Bin Wu, Xiao He, and Feifei Li. ShapleyIQ: Influence quantification by Shapley values for performance debugging of microservices. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 4, pages 287–323, 2024.
- [29] Fangzhou Liang, Yibin Shen, Jianmin Hu, Jiayang Xu, Hanchi Gao, Minxian Xu, and Zili Meng. SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC. *arXiv preprint arXiv:2609.18110*, 2026.
- [30] Yu Lin, Yiming Wang, Runyuan Cai, Hanze Liu, and Xiaodong Zeng. The other half of the memory wall: Serving 35B MoEs from SSD with trained routing prediction. *arXiv preprint arXiv:2609.18063*, 2026.
- [31] miltos22. llama.cpp pull request #26563: Expert caching that greatly increases performance (self-contained, off by default). <https://github.com/ggml-org/llama.cpp/pull/26563>, 2026. Accessed September 2026.
- [32] Niklas Muennighoff, Luca Soldaini, et al. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.
- [33] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [34] Xinyu Qiu, Chuhong Xu, et al. GGUF-metadata prediction of single-sequence llama.cpp throughput across three systems. *arXiv preprint arXiv:2609.14864*, 2026.
- [35] Anish Saxena, Po-An Tsai, Hritvik Taneja, Aamer Jaleel, and Moinuddin Qureshi. Cascade: Utility-driven speculative decoding for mixture-of-experts. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026. arXiv:2506.20675 (2025).
- [36] Musa Shams. Paging the experts: A reproducible characterization of flash-backed MoE inference on iPhone. *arXiv preprint arXiv:2609.29032*, 2026.
- [37] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116, 2023. arXiv:2303.06865.
- [38] Yuan Si, Yufeng Lin, et al. Who should own the expert cache? kernel-managed tiering for trillion-parameter MoE inference. *arXiv preprint arXiv:2608.12103*, 2026.
- [39] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [40] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [41] Shriniwas Ramesh Suram. Cacheable by design? training mixture-of-experts routers for locality against the edge memory-bandwidth wall: A pre-registered negative result, with a systems measurement study. *arXiv preprint arXiv:2608.18261*, 2026.
- [42] Peng Tang, Jiacheng Liu, et al. HOBbit: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [43] Aditya Ukarande, Deep Shekhar, Marc Blackstein, and Ram Rangan. Efficient, VRAM-constrained xLM inference on clients. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 8, 2026. Industry track. arXiv:2604.26334. Artifact: <https://github.com/deepshnv/pipeshard-mlsys26-ae>, tag v2.0.3-mlsys26.
- [44] Wenfeng Wang, Jiacheng Liu, Xiaofeng Hou, Xinfeng Xia, Peng Tang, Mingxuan Zhang, Chao Li, and Minyi Guo. MoE-SpeQ: Speculative quantized decoding with proactive expert prefetching and offloading for mixture-of-experts. *arXiv preprint arXiv:2511.14102*, 2025.
- [45] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.

- [46] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.
- [47] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [48] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [49] Sanjali Yadav and Bahar Asgari. Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging. *arXiv preprint arXiv:2609.38090*, 2026.
- [50] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [51] Shuo Yang, Xiaoze Fan, Melissa Pan, Haocheng Xi, Zhe Wang, Shanlin Sun, Kurt Keutzer, Song Han, Matei Zaharia, Chenfeng Xu, and Ion Stoica. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.
- [52] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- [53] Hanhaodi Zhang. Budgeting bytes: A windowed storage roofline and dual-budget architecture ablations for storage-bound LLM decoding. *arXiv preprint arXiv:2609.04238*, 2026.
- [54] Jiamu Zhang, Liang Wu, Mayank Darbari, and Liangjie Hong. WiSP: A working-set view of mixture-of-experts serving on extremely low-resource hardware. *arXiv preprint arXiv:2606.21868*, 2026.
- [55] Lei Zhang, Reza Karimi, Irfan Ahmad, and Ymir Vigfusson. Optimal data placement for heterogeneous cache, memory, and storage systems. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (PO-MACS)*, 4(1):6:1–6:27, 2020.
- [56] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
- [57] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference. In *Design, Automation & Test in Europe Conference (DATE)*, pages 1–7, 2025. *arXiv:2501.10375*.
- [58] Shuzhang Zhong et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. *arXiv:2504.05897*.
- [59] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.
- [60] Zeyu Zhu, Gang Li, Peisong Wang, Zitao Mo, Minnan Pei, Zhuoran Song, Xiaoyao Liang, and Jian Cheng. DALI: A workload-aware offloading framework for efficient MoE inference on local PCs. *arXiv preprint arXiv:2602.03495*, 2026.

A Scientific benchmarking checklist

How this paper follows the twelve rules of Hoefer and Belli [16].

Rule	Where and how
1. Report the base case of a speed-up and its absolute performance.	Every ratio in Table 2 is next to both absolute speeds; llama.cpp is configured with per-layer expert offload at equal expert memory, the setting its users run.
2. Explain subsets of benchmarks or resources.	Two models chosen for size and precision (MXFP4, BF16), a third that is FreeToken’s own headline model; budgets span what fits a 32 GB card; the 43.75% budget is omitted where it does not fit. All 30 AIME-25 problems are used.
3. Arithmetic mean for costs, harmonic mean for rates.	Every problem decodes the same number of tokens, so the harmonic mean of rates is total tokens over total time; ratios of total time agree with the reported ratios of mean rates within 0.003.
4. Avoid summarising ratios.	Ratios of means are never averaged across budgets or models; ranges list the per-budget values, and the medians reported across models (Table 4) or across published rows (Appendix H) are medians of per-cell quantities, labelled as such.
5. Report whether measurements are deterministic; give confidence intervals.	Decode is greedy and deterministic per engine; timing is not. Every comparison has a 95% interval.
6. Do not assume normality.	Intervals are bootstrap percentile intervals (10,000 resamples, paired by problem); no distributional assumption.
7. Compare nondeterministic data soundly.	Paired bootstrap of the ratio of means; the variant a system is scored with is picked on a separate launch (except the Qwen3 25% row of Table 2).
8. Check that central tendency is the right summary.	Mean speed over 256 tokens is FreeToken’s own metric; per-token p50 and p99 latencies are recorded for every problem in the released rows.
9. Document all factors and the setup.	Machines, probes, power limits, memory clocks (Section 2), software commits (llama.cpp 4da6337, FreeToken 0d652e7), flags and job scripts are released; Appendix I.
10. Report measurement, synchronisation and summarisation.	One client times every system of Table 2 from streamed token arrivals; the oracle comparisons of Section 4 are timed in process by llama-ec-bench over teacher-forced text, every policy by the same timer (Section 2).
11. Show upper performance bounds.	The speed limit (Section 3) and all-in-VRAM speed are columns of Table 2; Table 3 tightens the limit four ways; Section 3 applies it to the published literature.
12. Plot enough to interpret; connect points only for trends.	Figures 1, 4 and 6 are bar charts; Figs. 2, 3 and 7 are scatter plots; Fig. 5 connects measured windows for the trend only, and its W_{50} is a stated log interpolation between two measured windows.

B Prediction log

Each job’s predictions are in the header of its script on the public `gpu` branch, committed and pushed before the machine started; git history orders every prediction before its measurement. Outcome notes with the statistics are in `prereg/`. The scorecard, generated from `prereg/scorecard_clauses.json`, splits every prediction of jobs 073–098 into its separable clauses and scores each under one rule: *held* only when the point estimate is on the predicted side and the 95% paired-bootstrap interval excludes the threshold (for a band, lies inside it); *held (point)* when only the point estimate is, or no interval exists; *failed* when the point estimate is on the wrong side; *untested* when the clause could not be evaluated; *void* when the job was voided. Of 565 clauses, 218 held, 193 held on the point estimate, 142 failed, 8 were untested and 4 void: 7 of 19 scored clauses held in jobs 073–075, when the system was being built, 19 of 23 in jobs 076–081, 11 of them sign-only predictions made after a same-host pilot, 24 of 38 in jobs 082–087, and 168 of 473 in jobs 088 onward, where 42 of 82 sign clauses held (75 on the point estimate) and 90 of 301 bands failed. Most bands that failed are the sizes of effects whose direction held: our slow-link lead landed at $1.51\text{--}1.58\times$ against

a predicted 1.10–1.50; the rerun of Table 2 on a second host (job 089) gave ratios 0.06–0.11 below host B’s at five of six cells against a predicted ± 0.06 , host-bound speeds 14–17% lower against a predicted $\pm 12\%$, and LRU 6–8% slower than decayed frequency at the two GPU-heavy cells against a predicted 10–30%; on the 4090 and 3090 hosts (jobs 091, 092; Appendix D) our lead over FreeToken and its fraction of the limit overshoot their bands, FreeToken’s carried-over backend fell far below its band, and two launch-order checks exceeded 3% (7% and, at gpt-oss 11% on the 3090 host, 11%); in the first oracle job (093; Section 4) the oracle’s gain fell outside 20–80% at the two lowest budgets (it lost), its speed fell just below the predicted 45–65% of the limit at all four host-bound cells (43.7–44.8%), its hit rate and admissions missed the optimum’s by more than the predicted 3 points and 30% at three and two of the four, no-overlap cost 0% against a predicted 5–25%, and all-CPU cost 15% on Qwen3 against a predicted 0–10%; in the bytes-oracle job (094) the bands missed as Appendix F describes; in the single-read job (095) 44 of 56 clauses held, and the 12 that failed are of size: the law on the fetch oracle’s counters was predicted within 10% and missed by 13–17% at four cells, the fetch oracle’s reads were predicted within 10% of the simulation’s and came 17% below at the two GPU-bound cells (the registered simulator left a resident carried over from the previous sequence with no next use, and so evicted it first; corrected, the simulation is within 4% of the engine’s reads at every cell, Appendix F), the prefetching oracle was predicted at 55–85% of the limit at the host-bound cells and reached 44 and 51% at the two 25% cells, the law’s over-prediction of it was predicted at 15–50% and came to 7 and 13% at the two lowest budgets, and the paced variant, predicted within 10% of the unpaced one, was 12–13% faster at the two 25% cells; in the confirmatory factorial (096) the clauses held at most cells of both hosts and the failures are listed in Appendix F, the largest being the difference between the hosts; and in the FreeToken tuning job (098) its best setting stayed within the predicted 10% of its default and our ratios within the predicted bands, while the fixed fetch caps, the thread count and the CPU-only comparison missed as `prereg/freetoken.outcome_098.md` describes. Among the 23 threshold failures are ours at $1.56\text{--}1.78\times$ llama.cpp on the slow hosts against a predicted 1.8, the Mixtral tie against a predicted $1.3\times$, the hit-optimal oracle’s gain at the two GPU-bound cells (48 and 56%) against a predicted ceiling of 15%, the bytes-optimal oracle short of beating the hit-optimal one by 15% at the two lowest budgets, and its 16-token window short of keeping 70% of its gain at three cells, fetch’s gain differing between hosts O4 and O5 by more than 0.15 at five cells, Belady’s single-read prefetch above $1.05\times$ the online policy at Qwen3 12.5% on O4, and FreeToken’s CPU backend running close to stock llama.cpp’s where we predicted at least $2\times$ and $1.2\times$; the 3 equality failures are two of the 4090 host’s FETCH tables, which copy one more expert than predicted, and FreeToken’s best backend at gpt-oss 25% in job 098 (offload, not hybrid); the 7 sign failures are LRU behind FreeToken at gpt-oss 11%, where we predicted it would still lead, and the hit-optimal oracle slower than the online policy at gpt-oss 11% and Qwen3 12.5%. Against the running log kept during the study, two clauses are downgraded (a llama.cpp clause of job 074b that the log counted as held was never run; job 085b’s own restatement of its fourth prediction, “at least 2 of 3”, failed at 1 of 3, though the pooled 4 of 6 held), job 073’s “identical outputs” is scored on fully identical outputs (0–3%) rather than on agreement of the first 160 characters (27–47%), and ten clauses the log counted as held, among them job 087’s first, hold on the point estimate only under the interval rule; `prereg/scorecard.outcome.md` lists every change. Job 085’s Qwen3 half ran as 085b on a second machine with a slow link after a hung conversion. Job 084 ran on a throttled card and was rerun, with its predictions unchanged, as 084b/084c. The batched-verification variants of Section 5 (rejected drafts free, routed like the true token, or routed at random) are in `prereg/batchk.outcome.md`.

The clause-by-clause table (565 rows) is in the supplement (`paper/supplement.pdf`), generated by the same script.

C The law’s form and its failures

The blind test in full. On every machine of the blind test the job wrote the law’s prediction from its own probe before any model ran (Table 7; all of its cache rows are gpt-oss-120b on RTX 5090 hosts). Over 29 configurations on 7 machines (33 measurements; one machine ran twice) the median error is 3.7% per configuration; without the 12-channel EPYC 9655 it is 3.3% over 24 configurations (90th percentile 7.5%). llama.cpp `--n-cpu-moe` is predicted within 4% on the Ryzen hosts, including two Qwen3 configurations on host B (+3.6%, +4.0%; for Qwen3, $G=4.3$ ms, the mean residual of three llama.cpp runs on another host), and misses by +10–+119% on the hybrid-core and server hosts. With its constants frozen, the law scores the cache on the 4 RTX 5090 hosts rented later (30 measurements at the six cells of Table 2, not written before the runs) at a median 5.6%, 90th percentile 16%, worst 21% (Qwen3-30B-A3B 43.75% on oracle host O1). Refit under leave-one-host-out validation, the token-level max has a median error of 3.1% (90th percentile 9.4%), the additive form $G + X_c/B_c + X_p/B_p$ 4.5% (11.3%) and a per-layer max summed over layers 3.2% (10.2%); the max beats the sum on the three rows that copy misses on demand behind a 53 GB/s link, where both paths carry most of the bytes, and on the EPYC 9655 rows, where both fail, the sum is closer on the two rows that copy behind a slow link, and elsewhere the two are within a few points (Appendix C). Each failure has a measured cause (Appendix C): on-demand copies behind a slow link share DRAM less evenly than the probe’s

Table 7: The law predicted blind: each machine wrote its prediction from its own bandwidth probe, with constants frozen in the public repository, before any model run. Error of predicted over measured speed, gpt-oss-120b (cache rows) and llama.cpp --n-cpu-moe (last column, %).

Host CPU	Job	n	Median err %	Max err %	llama.cpp err %
Core Ultra 7 270K (8P+16E)	069c	5	1.5	11.9	+119
EPYC 7352 (Zen 2)	069c	5	3.2	11.7	+10
EPYC 9655 (12-ch. DDR5)	069c	5	18.7	31.9	+16
Ryzen 9 7900	072	5	4.9	8.2	+4
Ryzen 9 9950X3D (A)	073	8	1.6	4.5	—
Ryzen 9 7950X	074	1	4.0	4.0	—
Ryzen 9 9950X	076	4	2.5	5.0	—
Ryzen 9 9950X3D (B)	082	0	—	—	+4, +4

combined rate says; llama.cpp splits every matrix evenly across threads and runs at 46% of the law on a hybrid-core host; and on a 12-channel EPYC host memory stops being the limit and a per-layer latency floor takes over. G (5.17 ms refit) exceeds the whole all-in-VRAM token (3.92 ms) because it absorbs a shortfall the probe does not see: the helpers read at $1.09\text{--}1.31\times$ the probe’s time per expert, and a fitted rate factor is collinear with G at this sample size.

What the law says, and the split it picks. Equation (1) is a calibrated model of execution that reads host memory in sequence with its GPU work, as our cache and llama.cpp do. For such an engine every mechanism acts only through the bytes read from host memory per token and the number of read paths in use: a second path adds bandwidth on every host of the blind test but the 12-channel EPYC (B_{cp} against $\max(B_c, B_p)$): 8–17% on its Ryzen hosts, 1% on the Core Ultra, 33% on the Zen 2 EPYC; the RTX 3090 host’s Core i9-11900KF, where the two paths together read less than the CPU alone, is a second exception), and that is the whole gain available to fetching or splitting misses between the CPU and PCIe. Prefetching never reduces the bytes; what it can buy is overlap of the reads with the step, which the law does not price and Section 4 measures (there the law over-predicts the time per token of overlapped execution by up to 53%). The law’s job in the system is the split. Its per-layer form,

$$T(n, f) = \max\left(\frac{fS}{B_p} + g, [n > f](L_c + \frac{(n-f)S}{B_c}), [0 < f < n] \frac{nS}{B_{cp}}\right), \quad (3)$$

with g the GPU’s expert work per layer ($48\mu\text{s}$ for Qwen3, $37\mu\text{s}$ for gpt-oss, from profiles) and $L_c=20\mu\text{s}$ the hand-off, picks for each count n of missed experts the number f to copy with the smallest T . On host B the law’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0%; on hosts whose link we measured at 27–30 GB/s instead of 53 it fetches less and gains 21–34% on gpt-oss and 9–18% on Qwen3, the per-machine split mattering most where the link is slow (Appendix G). In the running example on host B the table is 0,0,1,1,2 ($f(0..4)$): one missed expert runs on the CPU, hidden behind the GPU’s other experts; from two misses the law copies one. The lead over FreeToken at that cell, $1.27\times$, is mostly the admission policy (77% of it by the LRU ablation of Appendix G), the rest the CPU path that FreeToken’s bandwidth model declines to use there.

Is the max the right form? In 19 of the 29 configurations nothing is copied, so they test one roof. We therefore refit the law and its alternatives on the same 33 measurements under leave-one-host-out validation (each of the 7 hosts predicted with constants fitted on the other six): the token-level max of Eq. (1) with G refit has a median error of 3.1% (90th percentile 9.4%); the additive form $G + X_c/B_c + X_p/B_p$, 4.5% (11.3%); a per-layer max summed over layers from the engine’s per-layer miss histograms, 3.2% (10.2%). The additive form ties in sample but misses the three rows that copy misses on demand behind a 53 GB/s link, where both paths carry most of the bytes, by -11%,

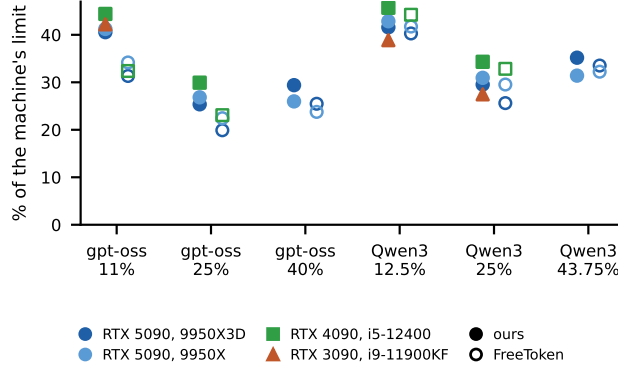


Figure 2: Ours (filled) and FreeToken (hollow) as a percentage of each machine’s own speed limit (exact optimum, the machine’s highest probed host rate, the card’s datasheet rate) at every cell of Table 8: two RTX 5090 hosts, an RTX 4090 and an RTX 3090 (FreeToken’s 3090 cells, its carried-over backend running below llama.cpp, are omitted). The limit ranges from 41 to 519 tok/s across these cells; ours stays at 25–46% of it.

while the max is within $\pm 0.2\%$ there; on the slow-link copy rows the additive form is the closer of the two. The per-layer form neither beats the max nor explains its failures: the Jensen gap between a sum of per-layer maxima and the maximum of sums is a fifth of the Core Ultra’s residual, none of the EPYC 7352’s, and has the wrong sign on the EPYC 9655 and on the prefetch rows. We keep Eq. (1), with its evidence stated: the max beats the sum on the three fast-link copy rows and on the EPYC 9655 rows, where both fail, the sum is closer on the two slow-link copy rows, and elsewhere the two are within a few points, so the form is settled only where both paths carry most of the bytes. FreeToken’s own runs are not predicted, because its counters are not exposed; the law covers our engine and llama.cpp.

Where it fails, and why. Each failure now has a measured cause. (i) Copying misses on demand (FETCH) at 25% is over-predicted by +11.7% on the Core Ultra 7 and +11.4% on the EPYC 7352 (links of 36 and 26 GB/s); +8.8% and +8.5% with G refit. The engine’s per-layer timers show why: a CPU miss served beside a copy takes 20–62% longer than one served alone, so the two paths share DRAM less evenly than the probe’s combined rate says; a variant of the law that uses the probe’s per-path rates under contention and a per-layer hand-off constant predicts both rows within +3.1% and +4.1%, at a worse median (4.2%). (ii) llama.cpp on a Core Ultra 7 with 8 performance and 16 efficiency cores runs at 46% of the law at its default thread count, because it splits every matrix evenly across its threads; our helpers claim work dynamically and stay on the law on the same machine. We did not test llama.cpp restricted to the performance cores. (iii) On a 12-channel EPYC 9655 (559 GB/s) host memory stops being the limit (+16 to +29% with G refit under leave-one-host-out validation; Table 7 gives the blind errors): the per-layer timers show a floor of 55–70 μ s per layer with CPU misses that no desktop host has (their intercepts are at or below zero), the latency of the hand-off at a bandwidth where a 13 MB expert takes 24 μ s. (iv) Prefetch configurations are under-predicted by -8.7 to -3.4%: their copies overlap attention, which the law charges in full.

D The grid: other cards, and the scope of the gain

Table 8 lists every cell of Table 2’s protocol on the four machines we measured, each against its own limit, and Fig. 2 plots the fractions. The protocol was repeated on an RTX 4090 next to a Core i5-12400 (the host reads 37 GB/s, the link 25) and an RTX 3090 next to a Core i9-11900KF (45 GB/s; CPU and link together read *less* than the CPU alone, so the law’s table copies nothing), at the cells that fit 24 GB. On the 4090 ours runs $1.03\text{--}1.37\times$ FreeToken and $1.8\text{--}2.6\times$ llama.cpp at half host B’s absolute speeds, the host being the reason; on the 3090 it runs $1.6\text{--}2.2\times$ llama.cpp (FreeToken’s carried-over backend ran below llama.cpp there and is not counted). The cards’ own bandwidth never enters: every one of the seven cells on the 24 GB cards is host-bound at the limit, and ours stands at 30–46% and 27–42% of it. Mixtral-8x7B (8 experts per layer, top-2; 26 GB of experts on the 3090) marks the scope: with 2 of 8 experts resident the cache hits 31% of reads where whole-layer pinning hits 25%, so there is little locality to earn, and ours ties llama.cpp (1.015 [1.008, 1.024]); on the 128-expert models the same cache hits 53–79% at 11–25% resident. Across eight hosts the gpt-oss lead over FreeToken grows with the host’s CPU-to-link ratio, from $1.09\text{--}1.22\times$ where the CPU reads $1.05\text{--}1.14\times$ its link’s rate to $1.51\text{--}1.58\times$ behind a link of half the bandwidth, in the direction the

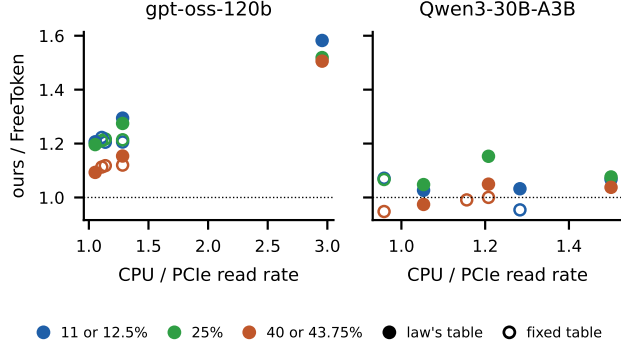


Figure 3: Ours over FreeToken’s better backend (carried over from host B on host S) in every same-machine comparison of the current engine on the two models of Table 2 (eight hosts), against the machine’s CPU-to-PCIe read ratio (FreeToken’s own probe). Filled: FETCH table from the law on that machine; hollow: the fixed table. The rightmost gpt-oss points are a Core i9-14900K with FreeToken’s executor on its performance cores; the rightmost Qwen3 points are a Ryzen 9 7900.

law’s split implies (the law does not predict FreeToken’s runs), and the Qwen3 lead stays at $0.95\text{--}1.15\times$ (Fig. 3, Appendix G).

The 24 GB cards run the cells that fit (the VRAM arithmetic, written on the machine from the GGUF header, skipped Qwen3 at 43.75%: 28.0 GB of slots, dense weights and KV against a 23 GB limit). The predictions for jobs 091 and 092 and their outcomes are in `prereg/grid_outcome_091.md` and `092.md`: on the 4090 our lead over FreeToken at gpt-oss 11% (37%) exceeded the predicted 10–35% band, ours ran $1.78\times$ llama.cpp at Qwen3 12.5% against a predicted 1.8, the law’s tables copied one more expert than host B’s at one entry each (the prediction reasoned from the link alone; the CPU is slower in the same proportion), and ours’ fraction of the limit at Qwen3 12.5% (45.7%) exceeded the predicted 25–45%; on the 3090 FreeToken’s carried-over backend ran below llama.cpp, ours ran $1.56\text{--}1.64\times$ llama.cpp at two cells against a predicted 1.8, Mixtral tied at $C=2$ against a predicted $1.3\times$ and its $C=4$ cell was lost to the job’s deadline, and one launch-order check exceeded 3% (7%).

E The accounting as a model

Between the measured time of a cell and its limit lie five modelled fixes: overlap of host reads with GPU work (in Eq. (2)’s form, which also rebalances experts onto the CPU), the optimum’s reads instead of the online policy’s (*foresight*), the best online policy’s reads at the probe’s highest rate instead of the engine’s through Eq. (1) (policy and read paths), the engine’s per-token host work, and GPU kernels at the datasheet rate. Adding them in a fixed order makes the first one added look largest, so we report each fix’s Shapley value, its average marginal cost over every order (Fig. 4), a method with a precedent in performance debugging of microservices [28]. Two of the five terms are definitions rather than measurements: the GPU term is the residual, its factor calibrated per cell so that the model with no fix applied reproduces the measured time, and “overlap” is Eq. (2)’s form. On host B the model has two regimes. In the four host-bound cells missing foresight is the largest cost, 37–53% of the gap (26–31% of the token; in the running example 37%, where it edges overlap by 0.14 ms and is the largest marginal cost in 70 of the 120 orders); in the two GPU-bound cells host reads serialised with GPU work is largest, 39–46% of the gap, and foresight second at 23%. Policy and read paths cost 12–16%, per-token host work 3–10%, and GPU kernels below datasheet, net of the overlap the engine achieves, 4–16%. The regimes are a property of the host: recomputed for the three oracle hosts, whose host memory is weaker, foresight edges overlap at three of their six GPU-bound cells. The model prices foresight and overlap as nearly independent players, and the interior states it values are not realised by any online policy. The rest of this section measures them.

F Foresight in the engine

Four jobs ran oracle policies in the engine, each on a rented RTX 5090 host with weaker host memory than host B’s. The fourth (096) is the confirmatory factorial of Fig. 1: ten configurations per cell, each in a fresh context, in an order shuffled per cell with a recorded seed, on hosts O4 (Ryzen 9 9950X: 43, 45 and 51 GB/s by the CPU, over

Table 8: The grid: Table 2’s protocol on every card and host we measured, each cell against its own machine’s limit (the exact optimum’s reads on the same trace, the machine’s highest probed host rate, the card’s datasheet rate). *Bound*: what binds the limit; *host* when the optimum’s reads alone take longer than the GPU’s whole read, so the GPU has slack, *both* when the limit runs experts on the CPU until the two paths take equally long, which is where the all-in-VRAM speed can be exceeded. Ours and FreeToken are launch 2 (FreeToken’s backend carried over from host B’s selection); the 40 and 43.75% budgets do not fit a 24 GB card. [†]On the RTX 3090 FreeToken’s carried-over hybrid backend ran below llama.cpp; its own calibration recommends its offload backend there, which we did not run, so its speed is reported and no ratio against it is printed. Mixtral-8x7B (26 GB of experts, top-2 of 8) is ours against llama.cpp only, on the RTX 3090, launch 1.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Limit (tok/s)	Bound	Ours, % of limit
RTX 5090, 9950X3D (host B): B_{host} 88 GB/s, B_{gpu} 1792 GB/s; tables 0,0,1,1,2 / 0,0,1,1,2,3,3,4,5									
gpt-oss-120b	11%	34.9	54.0	69.9	1.29 [1.28, 1.31]	2.00	172	host	41
gpt-oss-120b	25%	40.0	85.6	109.2	1.27 [1.25, 1.29]	2.73	430	host	25
gpt-oss-120b	40%	47.7	132.2	152.5	1.15 [1.13, 1.17]	3.20	519	both	29
Qwen3-30B-A3B	12.5%	19.5	38.7	40.0	1.03 [1.02, 1.04]	2.05	96	host	42
Qwen3-30B-A3B	25%	22.1	54.7	63.1	1.15 [1.13, 1.17]	2.86	214	host	30
Qwen3-30B-A3B	43.75%	28.4	103.1	108.2	1.05 [1.04, 1.06]	3.81	308	both	35
RTX 5090, 9950X (host S): B_{host} 71 GB/s, B_{gpu} 1792 GB/s; tables 0,0,1,2,3 / 0,0,1,2,2,3,4,5,6									
gpt-oss-120b	11%	28.9	48.0	57.9	1.21 [1.19, 1.22]	2.00	140	host	41
gpt-oss-120b	25%	33.6	78.7	94.1	1.20 [1.16, 1.22]	2.80	351	host	27
gpt-oss-120b	40%	40.1	122.2	133.6	1.09 [1.08, 1.11]	3.34	514	both	26
Qwen3-30B-A3B	12.5%	16.1	32.7	33.5	1.03 [1.02, 1.04]	2.08	78	host	43
Qwen3-30B-A3B	25%	18.5	51.4	53.9	1.05 [1.04, 1.06]	2.91	174	host	31
Qwen3-30B-A3B	43.75%	23.8	98.3	95.8	0.97 [0.96, 0.99]	4.02	305	both	31
RTX 4090, i5-12400 (job 091): B_{host} 37 GB/s, B_{gpu} 1008 GB/s; tables 0,0,1,2,2 / 0,0,1,1,2,3,4,4,5									
gpt-oss-120b	11%	18.1	23.8	32.7	1.37 [1.35, 1.39]	1.81	74	host	44
gpt-oss-120b	25%	21.0	42.5	55.0	1.30 [1.26, 1.33]	2.62	184	host	30
Qwen3-30B-A3B	12.5%	10.5	18.2	18.8	1.03 [1.02, 1.05]	1.78	41	host	46
Qwen3-30B-A3B	25%	12.3	30.0	31.3	1.05 [1.03, 1.06]	2.54	91	host	34
RTX 3090, i9-11900KF (job 092): B_{host} 45 GB/s, B_{gpu} 936 GB/s; tables 0,0,0,0,0 / 0,0,0,0,0,0,0,0									
gpt-oss-120b	11%	22.8	8.4 [†]	37.3	–	1.64	88	host	42
Qwen3-30B-A3B	12.5%	12.3	9.0 [†]	19.2	–	1.56	49	host	39
Qwen3-30B-A3B	25%	13.8	11.2 [†]	30.1	–	2.18	110	host	27
Mixtral-8x7B Q4_K_M $C=2$ (25%)		8.22	–	8.34	–	1.015 [1.008, 1.024]	–	–	–

the link and at best) and O5 (Ryzen 9 9950X3D: 47, 27 and 51 GB/s), with ten predictions per host committed before either started ([prereg/foresight_outcome_096.md](#)). They held at most cells. The failures: fetch’s gain above the predicted band at Qwen3 25% on O4 and different between the hosts by up to 0.26 against the 0.15 predicted; Belady’s single-read prefetch faster than predicted at Qwen3 12.5% on O4, with reads below the predicted 1.25–1.9× the MIN prefetch’s at Qwen3 12.5% on both hosts and at gpt-oss 11% on O4 (down to 1.09×); the bytes share at Qwen3 12.5% on O4 (60% of the gap against a predicted 25–55%); and the reads fetch-on-admit saves at gpt-oss 40% on O5, just above the predicted 22%. Reading the online policy’s admissions once came to 0.97–1.03× at the host-bound cells, at the bottom of the predicted 0.97–1.12× although its reads fell as predicted, and to 0.95–1.02× at the GPU-bound cells. Table 9 lists every cell of the single-read factorial of Section 4 on host O3 (a Ryzen 9 9950X whose probe reads 53 GB/s by the CPU at the helper count, 43 over the link, 60 with both and 61.5 at best; the law’s FETCH tables 0,0,1,2,2 for gpt-oss and 0,0,1,1,2,3,4,4,5 for Qwen3), and Table 10 the milliseconds per token of the engine states the three jobs realised beside each host’s limit and the model’s values. Table 11 lists the first job (093), the hit-optimal oracle with a window sweep on host O1 (a Ryzen 9 9950X: 44, 46, 49 and 50.5 GB/s; tables 0,0,1,2,3 and 0,0,1,2,3,4,5,6,7), and Table 12 the second (094), MIN with bypass with background admissions beside the hit-optimal oracle on host O2 (a Ryzen 9 7950X: 44, 46, 50 and 50.6 GB/s; the same tables). On O1 and O2 every cell but gpt-oss 40% and Qwen3 43.75% is host-bound at the limit; on O3, whose memory is stronger, the same four.

The single-read oracles. In the fetch oracle the step’s plan is computed before the layer runs from the trace’s record of the current step: the misses MIN with bypass would admit are fetched by the engine’s FETCH operation into their victims’ slots and run from there, and every other miss runs on the CPU; the oracle refuses an admission when

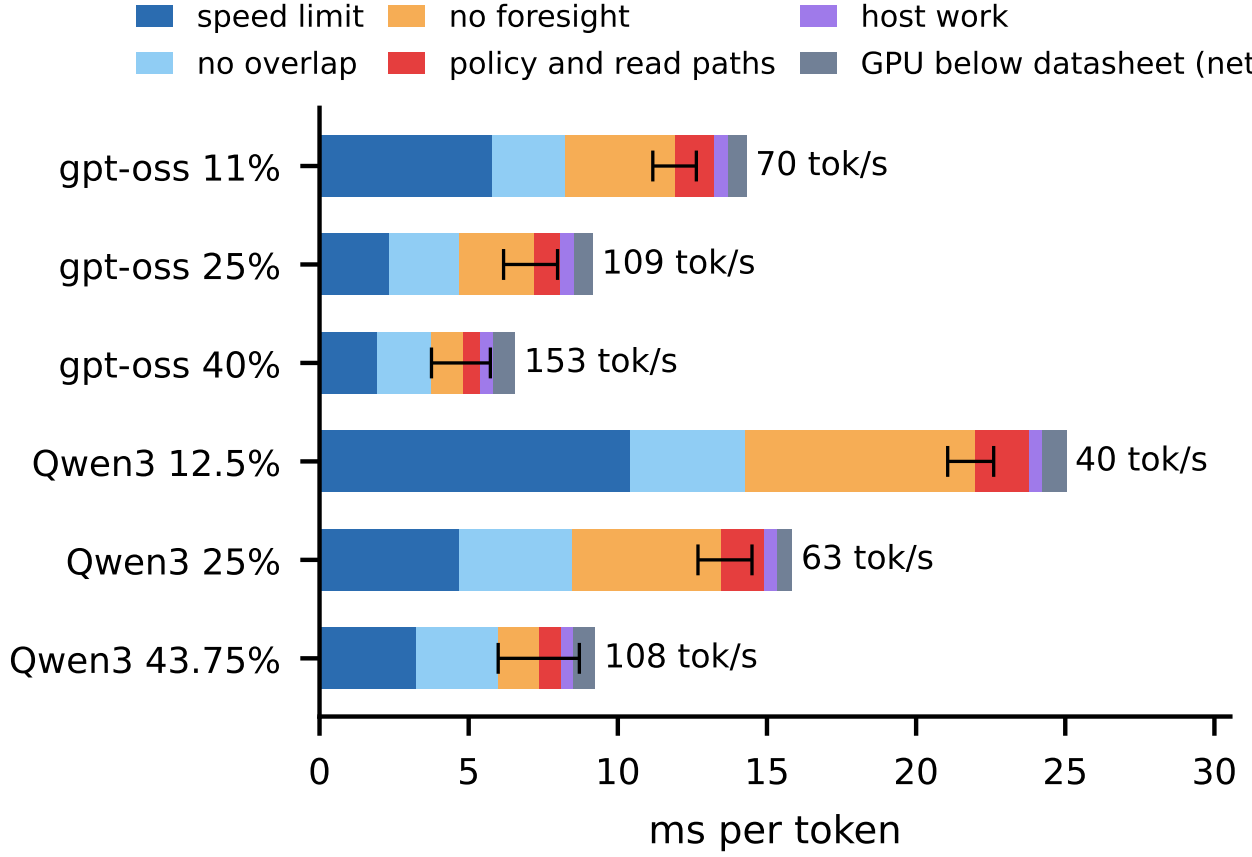


Figure 4: The accounting as a model: Shapley attribution of the gap between our cache’s measured time per token and the speed limit, for every cell of Table 2 on host B, averaged over all 120 orders in which the five fixes can be applied. Fixes: overlapping host reads with GPU work, which in Eq. (2)’s form also moves experts onto the CPU until the two paths balance; the optimum’s reads instead of the best online policy’s (foresight); the best online policy’s reads at the probe’s highest rate instead of the engine’s reads through Eq. (1) (policy and read paths); the engine’s per-token host work; and GPU kernels at the datasheet rate, the term that closes the model on the measured time. Whiskers: the range of foresight’s marginal cost over the orders.

no resident’s next use is further than the candidate’s. The single-read prefetch admits, each step, the non-resident experts whose first use is at least d steps away ($d=2$ unpaced, 3 paced), provided the victim’s next use is further than the candidate’s use after that first one; an expert used sooner than d steps away misses on the CPU, or is fetched when the two are combined. The same policies simulated on the trace with the engine’s semantics (the lookahead within the sequence, residents carried over with their next use recomputed at each sequence start) predict the engine’s reads and hit rates: the fetch oracle’s hit rate within 0.4 points and its reads within 0 to 4%, the combined oracle’s hits within two points (Table 9). The simulator the job’s predictions were registered against gave a resident carried over from the previous sequence no next use, so it was evicted first; that over-counted the fetch oracle’s reads by 17% at the two GPU-bound cells, which is why two of its reads clauses failed (Appendix B). Each configuration of `llama-ec-bench` starts with a cold cache in a fresh context. The paced copy path copies one 16 MB piece at a time on a single copier thread and publishes a landed copy at the start of the next step; the unpaced path copies each admission as one transfer. The hit-optimal oracle’s background admissions drive host memory at 71–82% of the probe’s best rate at the host-bound cells, the prefetching single-read oracles at 64–85% (the paced one 71–82%), the online policy itself at 66–82%, the fetch oracle at 42–67%. The predictions and their outcomes are in `prereg/foresight_outcome_095.md`: of 56 clauses, 44 held and 12 failed, all of size (Appendix B).

Table 9: The single-read oracles (job 095; RTX 5090, Ryzen 9 9950X, host memory 61.5 GB/s at best) beside the hit-optimal one. Per cell: the online policy’s speed and its share of this host’s limit; for each oracle with the whole sequence in view, its ratio to the online policy (paired, 95% interval), hit rate, host reads per token (CPU misses + fetches + admissions) as a multiple of the optimum’s (each read counted once), share of the limit and the law on its own counters against its measured time. *fetch*: MIN with bypass, admitted misses fetched into their slot this step; *lead*: the scheduled single-read prefetch (first use at least two steps away); *both*: fetch and lead; *paced*: both on the paced copy path with a three-step lead; *hit-opt.*: the hit-optimal oracle (Belady prefetch without bypass, unpaced). The simulation columns give the trace’s prediction of reads and hits for fetch and both.

Cell	online			fetch			law	both			hit-opt.			lead	paced		simulated		
	tok/s	% lim.	ratio	hits	reads/R*	% lim.		ratio	hits	reads/R*	% lim.	ratio	hits		reads/R*	% lim.	ratio	% lim.	ratio
gpt-oss 11%	55.5	46	1.29 [1.28, 1.29]	72	1.04	59 -8%	1.39 [1.39, 1.40]	86	1.23	64 0.91	[0.90, 0.92]	98 1.77	42 1.26	58 1.45	67 1.45	39 73	46 87		
gpt-oss 25%	90.3	30	1.32 [1.31, 1.33]	89	1.08	39 -13%	1.49 [1.48, 1.50]	97 1.45	44 1.14	1.12	[1.12, 1.15]	99 2.11	34 1.45	43 1.66	50 2.11	16 89	22 99		
gpt-oss 40%	128.8	25	1.25 [1.24, 1.27]	95	1.14	32 -14%	1.39 [1.37, 1.41]	99 1.60	35 1.27	1.25	[1.25, 1.28]	99 2.20	32 1.38	35 1.49	38 7.95	12 100			
Qwen3 12.5%	32.3	48	1.34 [1.34, 1.35]	74	1.04	64 -9%	1.42 [1.41, 1.43]	84 1.20	68 0.69	0.68	[0.71, 0.72]	77 2.47	33 1.22	58 1.41	68 100 74	115 84			
Qwen3 25%	51.8	34	1.38 [1.37, 1.39]	88	1.04	48 -15%	1.48 [1.46, 1.49]	96 1.42	51 1.06	1.05	[1.08, 1.10]	99 2.00	37 1.41	48 1.67	57 45	88 61	97		
Qwen3 43.75%	91.4	30	1.26 [1.25, 1.28]	96	1.08	38 -17%	1.41 [1.39, 1.42]	99 1.62	42 1.27	1.26	[1.29, 1.30]	100 2.29	38 1.40	42 1.50	45 15	96 23	100		

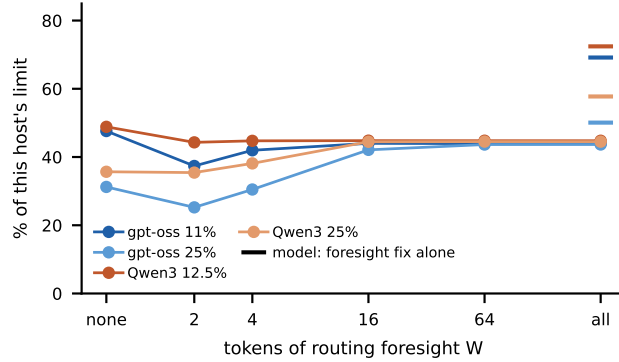


Figure 5: The first oracle job (093, host O1): speed as a percentage of this host’s limit against the window W of future routing the hit-optimal oracle sees, at the four host-bound cells. “none” is the online policy; “all” the whole sequence; points are connected for the trend only. The tick at the right is the model’s foresight-only term for the cell, the speed it predicts with that one fix; the oracle’s host traffic sits at 89–94% of the probe’s best rate at every window, so its speed is R^* over its reads times that utilisation, 44–45% of the limit with the whole sequence in view.

The double-reading oracles. Both read the routing of the next W decode steps; their copies ride the paced admission path, and an expert still in flight at its step runs on the CPU like any other miss. The *hit-optimal* oracle admits the experts those steps will route to, soonest first, evicting the resident whose next use is furthest (Belady within the window), for one future use as readily as for ten. The *bytes-optimal* oracle is MIN with bypass with its admissions made through the background path: a miss is served where it is, by the CPU or a demand copy, and admitted only if its next use comes before the furthest next use among the residents. With the whole sequence in view the hit-optimal oracle runs $1.40\times [1.38, 1.41]$ the online policy at gpt-oss 25% and $1.25\times [1.24, 1.26]$ at Qwen3 25% on host O1, $1.63\times$ and $1.41\times$ on O2, and 1.48 – $1.62\times$ at the two GPU-bound cells; half of its gain at gpt-oss 25% arrives between $W=4$ and 16 (9.2 tokens by log interpolation; Qwen3 25%: 6.1 against the trace’s 3.9). Where the host binds hardest it loses, 3–8% at gpt-oss 11% and Qwen3 12.5% on both hosts, and at every window on O1 (0.78 – $0.92\times$): it reads more host bytes than the online policy (82 and 195 experts per token against 63 and 160), 2.02 – $2.18\times$ the optimum’s R^* , because Belady admits an expert used once where the optimum bypasses it and re-copies what it evicts (its admissions, each one read, are most of its reads; a miss served on the CPU before its copy lands is the second read of that expert). An unpaced copy path, run at the four host-bound cells of O1, reaches 98–99% hits at three of them and still loses at gpt-oss 11% and Qwen3 12.5% (where it hits 75%), so copies that arrive in time do not remove the loss there: the extra bytes cause it. The bytes-optimal oracle gains 13–23% at the four cells of 25% and above and 0–3% at the two below (6% with a 16-token window); its misses fall (30 to 20 and 85 to 64 per token at 25%) but its total reads, misses plus the admissions that read each admitted expert again, are 1.73 – $2.04\times R^*$ and exceed the online policy’s by 4–5% at the two lowest budgets; the law on its own counters predicts its time within 9% at every cell, and the bytes it saves account for 4–21 points of its 13–23% gain at the four cells (most of it at gpt-oss 40% and Qwen3 43.75%, a third at Qwen3 25%). The hit-optimal oracle reads 1.01 – $1.13\times$ the bytes-optimal one’s host bytes on O2 and gains 41–63% at the same four cells, and the law, with its GPU term calibrated on the online

Table 10: The accounting, measured: milliseconds per token of the engine’s states that the oracle jobs realised, on each oracle host, beside that host’s limit and the model’s values for the states it prices; O1–O3 ran jobs 093–095 in a fixed order, O4 and O5 job 096 in a shuffled order. *online*: the deployed policy; *foa*: the same with each admission fetched into its slot in the step that misses it (one read, no foresight); *no ovl.*: its within-layer overlap off; *all CPU*: no FETCH table; *bypass*: MIN with bypass with background admissions (two reads per admitted expert); *hit-opt.*: Belady prefetch without bypass, which admits what the optimum bypasses, unpaced, and the same on the paced copy path (the first *paced* column; O1 and O2 ran only that form); *nb2*: Belady within a two-step lead with each admission read once; *fetch*: MIN with bypass, admitted misses fetched into their slot (one read, serialised: the model’s foresight-only state $v(F)$); *both*: fetch and the scheduled single-read prefetch (one read, overlapped: the model’s $v(O, F)$); the second *paced*: both on the paced copy path with a three-step lead; *limit*: the limit on that host; $v(F)$, $v(O, F)$, $v(\text{all})$: the model.

Host	Cell	online	foa	no ovl.	all CPU	bypass	hit-opt.	paced	nb2	fetch	both	paced	limit	$v(F)$	$v(O, F)$	$v(\text{all})$
O1	gpt-oss 11%	21.12	–	21.01	22.91	–	–	22.88	–	–	–	–	10.06	14.54	11.27	10.06
O1	gpt-oss 25%	12.89	–	12.91	14.21	–	–	9.22	–	–	–	–	4.03	8.04	5.10	4.03
O1	gpt-oss 40%	8.90	–	8.95	–	–	–	5.99	–	–	–	–	1.97	5.75	3.84	1.97
O1	Qwen3 12.5%	36.92	–	36.91	43.29	–	–	40.29	–	–	–	–	18.04	24.90	19.29	18.04
O1	Qwen3 25%	22.73	–	22.76	26.60	–	–	18.22	–	–	–	–	8.11	14.04	9.22	8.11
O1	Qwen3 43.75%	12.71	–	12.67	–	–	–	8.12	–	–	–	–	3.32	8.25	5.51	3.32
O2	gpt-oss 11%	20.85	–	–	–	20.17	–	21.57	–	–	–	–	10.04	14.27	11.96	10.04
O2	gpt-oss 25%	12.72	–	–	–	10.77	–	7.80	–	–	–	–	4.02	7.86	5.86	4.02
O2	gpt-oss 40%	8.79	–	–	–	7.13	–	5.62	–	–	–	–	1.97	5.63	3.78	1.97
O2	Qwen3 12.5%	36.64	–	–	–	36.58	–	38.74	–	–	–	–	18.00	24.70	20.06	18.00
O2	Qwen3 25%	22.24	–	–	–	19.67	–	15.80	–	–	–	–	8.09	13.58	9.90	8.09
O2	Qwen3 43.75%	12.02	–	–	–	10.11	–	7.42	–	–	–	–	3.32	7.56	4.90	3.32
O3	gpt-oss 11%	18.01	–	–	–	–	19.79	–	–	14.00	12.91	12.39	8.26	12.63	9.49	8.26
O3	gpt-oss 25%	11.08	–	–	–	–	9.74	–	–	8.42	7.44	6.66	3.31	7.05	4.28	3.31
O3	gpt-oss 40%	7.77	–	–	–	–	6.14	–	–	6.20	5.58	5.20	1.95	5.24	3.56	1.95
O3	Qwen3 12.5%	30.95	–	–	–	–	44.54	–	–	23.02	21.78	21.89	14.81	20.35	16.06	14.81
O3	Qwen3 25%	19.32	–	–	–	–	18.17	–	–	14.01	13.09	11.58	6.66	11.68	7.76	6.66
O3	Qwen3 43.75%	10.94	–	–	–	–	8.59	–	–	8.66	7.78	7.28	3.30	7.24	4.76	3.30
O4	gpt-oss 11%	20.59	20.18	–	–	20.11	19.78	22.03	19.64	15.13	14.06	13.78	10.02	13.77	11.63	10.02
O4	gpt-oss 25%	12.43	12.07	–	–	10.55	9.76	8.13	9.61	8.72	7.65	6.87	4.01	7.40	5.34	4.01
O4	gpt-oss 40%	8.39	8.19	–	–	6.91	6.15	5.38	6.03	6.28	5.59	5.21	1.97	5.12	3.20	1.97
O4	Qwen3 12.5%	36.36	35.83	–	–	36.54	48.80	39.66	31.64	25.29	24.01	24.58	17.97	23.98	19.88	17.97
O4	Qwen3 25%	22.09	21.75	–	–	20.06	17.74	16.93	17.57	14.60	13.50	12.24	8.08	13.12	9.54	8.08
O4	Qwen3 43.75%	11.94	11.74	–	–	10.09	8.44	7.17	8.33	8.69	7.72	7.29	3.32	7.32	4.56	3.32
O5	gpt-oss 11%	21.26	21.31	–	–	21.47	33.34	25.57	33.03	18.34	17.97	16.39	9.94	15.22	11.20	9.94
O5	gpt-oss 25%	13.01	13.35	–	–	11.18	15.87	11.17	15.60	10.60	10.54	8.68	3.98	9.13	5.06	3.98
O5	gpt-oss 40%	8.73	9.22	–	–	7.16	8.34	6.25	8.13	7.27	6.87	5.91	1.97	6.26	3.62	1.97
O5	Qwen3 12.5%	37.63	37.42	–	–	40.11	69.42	44.89	49.11	31.37	31.27	30.01	17.82	27.95	19.27	17.82
O5	Qwen3 25%	23.24	23.27	–	–	21.12	30.78	21.76	30.47	18.51	19.64	16.03	8.01	16.77	9.15	8.01
O5	Qwen3 43.75%	12.49	12.92	–	–	10.32	12.08	9.00	11.86	10.21	9.61	8.23	3.31	8.94	5.02	3.31

run, over-predicts those runs’ time per token by 37–53% (on O1, 24–39%): its gain is the overlap of copies issued ahead of their use with the step, which the law charges in sequence. Pacing the online policy’s own copies changes it by under 1% (0.997–1.005 $\times$). With no FETCH table (every miss on the CPU) the online policy runs 8–15% slower at the four host-bound cells; with the engine’s within-layer overlap of a layer’s CPU misses with its GPU hits turned off it changes by under 1% at every cell (0.5% faster to 0.6% slower), so the overlap the accounting charges is across the step and in rebalancing, not within a layer. The bytes-optimal oracle with a 16-token window keeps 34–176% of the whole-sequence gain at the five cells where that gain’s interval excludes zero (more than all of it at gpt-oss 11%, where the shorter window admits less).

Why the bytes-optimal oracle’s misses stay above the optimum’s reads. Besides the second read, its misses alone are 1.31–1.47 $\times$ the optimum’s reads at the four host-bound cells (at the two GPU-bound ones the ideal simulation below is itself 6–10% above the optimum, which sees across sequences). The engine’s paced copy path moves one 16 MB piece at a time from pageable host memory on a single copier thread, behind the demand copies, publishes a landed copy at the start of the next step, and admits nothing while 64 copies are queued; at the 16–30 admissions per token (210–285 MB) of the two lowest budgets it lags the admissions by steps, and an expert still in flight is a miss the CPU reads again. The last four columns of Table 12 simulate MIN with bypass on the measured routing trace with the engine’s own semantics (the lookahead within the sequence, residents carried over, no queue cap) and a copy that lands d steps after it is issued, during which the expert misses and is not re-admitted: at $d=1$, the ideal, the misses come within 0–4% of the limit’s optimum at the host-bound cells; the engine’s misses meet the simulation at $d=2.8$ –4.7 there and at 3.0–3.5 at the GPU-bound cells, three to five steps. At the fitted latency the simulation admits 1.1–1.9 $\times$ what the engine admitted, so the queue cap, not only the latency, keeps the engine’s misses above the ideal. The predictions and their outcomes are in `prereg/foresight_outcome_093.md` and `094.md`:

Table 11: The hit-optimal oracle in the engine (job 093; RTX 5090, Ryzen 9 9950X, host memory 50.5 GB/s at best). tok/s of the online policy and of the oracle with the whole sequence in view; their ratio, paired by sequence, with its 95% interval; hit rate and misses per token of each (experts read from host memory by the CPU or by a demand copy), the optimum’s reads in parentheses, and the oracle’s background admissions (Belady prefetch without bypass: it admits what the optimum bypasses); speed as a percentage of this host’s limit; the accounting’s foresight-only term $v(F)$ for the cell in tok/s; the share of that term the oracle recovered and the share of the gap to the limit it closed.

Cell	online	oracle	ratio	hits % (opt.)	misses/token (opt.)	oracle adm.	% of limit	$v(F)$	recovered %	closed %
gpt-oss 11%	47.4	43.7	0.92 [0.91, 0.93]	58 → 67 (73)	60 → 47 (38)	35	47.6 → 44.0	68.8	−27	−16
gpt-oss 25%	77.6	108.5	1.40 [1.38, 1.41]	79 → 92 (89)	30 → 11 (15)	20	31.2 → 43.7	124.4	76	41
gpt-oss 40%	112.4	166.9	1.48 [1.46, 1.51]	89 → 98 (95)	15 → 3 (7)	12	22.1 → 32.8	174.0	92	42
Qwen3 12.5%	27.1	24.8	0.92 [0.91, 0.92]	60 → 66 (75)	155 → 132 (97)	63	48.9 → 44.8	40.2	−28	−18
Qwen3 25%	44.0	54.9	1.25 [1.24, 1.26]	78 → 89 (89)	85 → 42 (43)	46	35.7 → 44.5	71.2	52	31
Qwen3 43.75%	78.7	123.1	1.56 [1.53, 1.59]	92 → 99 (96)	32 → 5 (14)	25	26.1 → 40.8	121.3	103	49

Table 12: The bytes-optimal oracle (MIN with bypass) beside the hit-optimal one on the second oracle host (RTX 5090, Ryzen 9 7950X, host memory 50.6 GB/s at best). Per cell: the online policy’s speed; each oracle’s ratio to it with the whole sequence in view (paired, 95% interval), its hit rate, its misses per token (experts read from host memory by the CPU or by a demand copy; its background admissions in parentheses: the bypass oracle’s are this step’s CPU-served misses, each read a second time by the copy, the hit-optimal one’s are prefetches of what the optimum would bypass as readily as of what it would admit) and the law on its own counters against its measured time; the optimum’s reads and hit rate from the trace; and MIN with bypass simulated on the trace with the engine’s lookahead (within the sequence) under a copy latency of 1 (ideal), 2 and 3 steps, with the latency at which the simulation meets the engine’s misses.

Cell	online	ratio	bypass oracle, W=all				hit-optimal oracle, W=all				optimum		simulated misses/token				
			hits	misses	(adm.)	law	ratio	hits	misses	(adm.)	law	reads	hits	d=1	2	3	fit
gpt-oss 11%	48.0	1.03	[1.03, 1.04]	65	50 (16)	+7%	0.97 [0.96, 0.98]	71	42 (39)	+19%	38.3	73	39	43	47		3.9
gpt-oss 25%	78.6	1.18	[1.18, 1.19]	86	20 (9)	+6%	1.63 [1.61, 1.66]	95	7 (23)	+53%	15.3	89	16	17	19		4.7
gpt-oss 40%	113.8	1.23	[1.22, 1.24]	94	9 (4)	+2%	1.56 [1.54, 1.59]	98	3 (12)	+39%	6.8	95	7	8	9		3.5
Qwen3 12.5%	27.3	1.00	[1.00, 1.01]	64	138 (30)	+5%	0.95 [0.94, 0.95]	67	128 (65)	+11%	96.5	75	97	122	143		2.8
Qwen3 25%	45.0	1.13	[1.12, 1.14]	83	64 (21)	+9%	1.41 [1.39, 1.43]	91	34 (52)	+37%	43.4	89	44	54	61		3.4
Qwen3 43.75%	83.2	1.19	[1.18, 1.20]	95	19 (9)	+1%	1.62 [1.58, 1.65]	99	3 (27)	+47%	13.7	96	14	17	19		3.0

of 36 clauses for the first job, 9 held, 2 held on the point estimate, 21 failed and 4 were untested; of 38 for the second, 12 held, 1 on the point estimate, 24 failed and 1 was untested (Appendix B). The second job’s failures are of size, not direction: the bypass oracle’s misses were predicted within 15% of the optimum’s reads and came 29–47% above (the copy latency and the queue cap); its hit rate was predicted within 4 points of the optimum’s and fell 5–11 points short at three cells; its gain was predicted at 10–45% at the host-bound cells and came below that at the two lowest budgets; the law was predicted to over-predict it by 10–40% at the host-bound cells and over-predicted by 5–9%; and the two oracles were predicted within 15% of each other at 25% and above, where the hit-optimal one was 1.24–1.38× faster.

G The expert cache in more detail

This section holds the measurements of Section 2 that the main text summarises: where the cache’s gain comes from, the choice of the FETCH split, the hosts with half the link, long outputs, FreeToken’s own headline model, two other llama.cpp-based systems, output parity, and the cost of prompts and batches. Job numbers refer to the scripts on the gpu branch.

Where the gain comes from. Figure 6 builds the system up from stock llama.cpp on one machine. Placing each layer’s hot experts is the first gain: a static cache of the 32 experts per layer most requested on other text (the models’ own MATH-500 outputs) runs 1.31× and 1.89× stock, which pins whole layers instead. Its hit rate (52% and 67%) depends on how well the profile matches the workload: profiled in hindsight on the measured problems it hits 75% and 72% and runs faster (hatched). Replacing experts online closes that gap without a profile, if admissions are rationed: LRU admits every miss, and on Qwen3 its copies raise host reads from 127 to 188 experts per token, so it runs 0.88× the static cache (1.30× on gpt-oss); the decayed-frequency policy reads 41 and 118 per token, runs 1.58× and 1.12× the profiled static cache, and the hindsight cache runs 0.98 [0.95, 1.01] and 1.01 [0.98, 1.04] times as fast as it. On traces LRU misses only 2–11% more than decayed frequency at these budgets (Section 5); the rest of its cost here is the copies it admits. On host S, with everything else as in the full system, LRU runs 6–21% slower than decayed

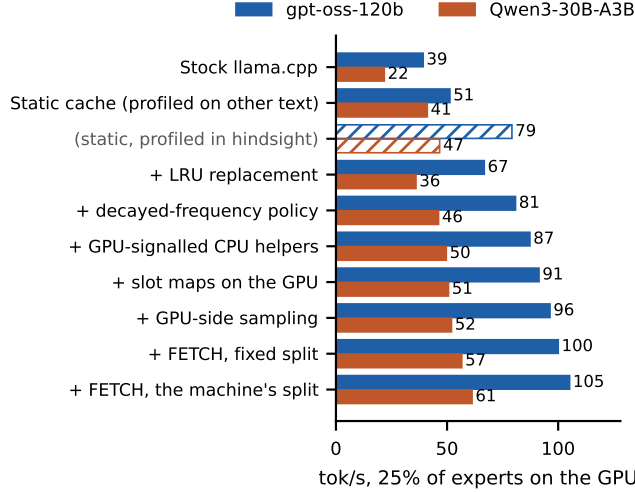


Figure 6: Ablation at 25% of experts on the GPU (15 AIME-25 problems, 256 tokens, host A). Each step adds one component to the one above; the hatched row is not a step: a static cache of each layer’s 32 most requested experts counted on the measured problems themselves.

Table 13: FETCH tables at 43.75% on Qwen3-30B-A3B (launch 1): entry n is how many of a layer’s n missed experts are copied over PCIe; the rest run on the CPU. Ratio to the fixed table, paired, 95% CI.

Table	$f(0..8)$	tok/s	vs fixed
Lite	0,0,1,1,1,2,2,3,3	109.0	1.057 [1.052, 1.062]
Law	0,0,1,1,2,3,3,4,5	108.0	1.048 [1.041, 1.055]
Law, fetch a single miss	0,1,1,1,2,3,3,4,5	104.1	1.010 [1.007, 1.012]
Fixed	0,1,1,2,3,3,4,5,6	103.1	1
None (CPU only)	0,...,0	100.7	0.977 [0.970, 0.984]
All (fetch every miss)	0,1,...,8	98.7	0.957 [0.953, 0.962]

frequency across the six cells of Table 2 and trails FreeToken at four of them: the admission policy is most of the lead over FreeToken (77–84% of it at gpt-oss 25 and 40%, more than the whole of it at gpt-oss 11% and Qwen3 12.5 and 25%). The GPU-signalled CPU helpers, GPU slot maps, GPU sampling and FETCH add 2–9% each, and the law’s FETCH table 5–8% over the fixed one. The full system runs $2.68\times$ (gpt-oss) and $2.81\times$ (Qwen3) stock llama.cpp.

Choosing the split per machine. On host B, the law’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0% (launch 1). Table 13 shows why at one budget: a single missed expert should run on the CPU (about $155\mu s$, hidden behind the GPU’s other experts) rather than be copied (about $225\mu s$, on the critical path); that entry alone is worth 3.7%. Fetching everything, FreeToken’s offload strategy expressed in our engine, is the worst table, and running everything on the CPU the second worst. The law’s table was within 1% of the best of five alternatives on launch 1 and tied with it on launch 2. Given each run’s measured misses, Eq. (3) ranks the tables correctly except law against lite, which are within 1%, and overstates their differences about $1.8\times$.

Machines with half the link. On RTX 5090s whose links we measured at 27–30 GB/s instead of 53 (jobs 085, 085b and 088: gpt-oss next to a Core i9-14900K, Qwen3 next to a Ryzen 9 7900 in a PCIe 4.0 slot), the law’s tables fetch less (gpt-oss 0,0,0,1,1, Qwen3 0,0,1,1,2,2,3,3,4) and beat the fixed table by 21–34% on gpt-oss and 9–18% on Qwen3, about twice the gain on host B for Qwen3 and four to six times for gpt-oss: the per-machine split matters most where the link is slow. On the Ryzen, ours leads FreeToken by 1.04 – $1.08\times$; with the fixed table it would trail

at 12.5%. On the Core i9 the first comparison (job 085) was confounded: FreeToken’s hybrid backend ran gpt-oss at 12–66 tok/s, below its own offload backend (25–86), because its CPU executor spread 23 threads across the 8 performance and 16 efficiency cores and its calibration then sent two thirds of the misses over the slow link, the same failure Appendix C reports for llama.cpp; the 1.4–2.0 $\times$ lead that gave us measured a thread configuration. We reran it (job 088, a second rental of the same CPU model) with FreeToken’s executor on 8 threads pinned to the performance cores: its hybrid backend then runs 1.4–3.1 $\times$ faster than before (37.5, 62.1 and 92.5 tok/s; its calibration now copies 51.7% of misses), beating its own offload backend at 11 and 25% and tying it at 40%, and ours leads its better configuration by 1.58, 1.52 and 1.51 $\times$ (lowest interval bound 1.47). Restricted to the CPU alone, FreeToken’s 8-thread executor decodes 13.7 tok/s at every budget. llama.cpp on the same rental runs 40.0 tok/s with one thread per performance core (−9.4% from the law’s prediction) and 32.8 with all 24 threads (+25.9%): the hybrid-core failure is a thread-count choice, and the law holds once the choice is right. The law predicted llama.cpp on the Ryzen within +2.7%. Figure 3 puts every same-machine comparison on one axis.

Long outputs. Our lead does not come from the first 256 tokens. On gpt-oss, on ten held-out hard problems (AIME 2022, problems 11–15 of both exams) that neither system has finished reasoning about after 2,048 tokens (at most 1 of 10 reach an answer), ours runs 1.36 [1.34, 1.38], 1.32 [1.30, 1.34] and 1.19 [1.17, 1.21] times FreeToken at 11, 25 and 40% over all 2,048 tokens, and 1.37 [1.34, 1.39], 1.33 [1.31, 1.35] and 1.19 [1.17, 1.22] over tokens 257–2,048 (job 083, host A); both engines stream one event per token. On Qwen3, over tokens 257–2,048 of ten held-out MATH-500 problems, ours runs 1.043 [1.020, 1.070], 1.056 [1.036, 1.077] and 1.054 [1.025, 1.086] times FreeToken at 12.5, 25 and 43.75%. Both leads are larger than over the same runs’ first 256 tokens. Turning off FreeToken’s prefill overlap, which lends it cache slots during each prompt, changes its overall speed by 1% or less.

FreeToken’s own headline model. On FreeToken’s own headline model, Qwen3.6-35B-A3B in BF16 (256 experts of 6.3 MB per layer, hybrid linear attention), the two systems are close: ours runs 1.030 [1.023, 1.038] times FreeToken at 12.5% and ties at 25 and 37.5% (1.010 [0.998, 1.021], 1.003 [0.994, 1.013]), at 1.98–2.48 $\times$ llama.cpp (job 086, host A). FreeToken as shipped, with its automatic backend and cache size, runs this model at 93.8 tok/s on that machine, above the 77–83 tok/s its paper reports on an RTX 5090, so our setup does not handicap it; FreeToken’s fetch cap and thread count were left at their defaults, while our FETCH table was swept. Our lead is therefore model-dependent: largest with few large experts (gpt-oss, top-4 of 128), smallest with many small ones.

FreeToken tuned. On an RTX 5090 with a Ryzen 9 9950X (job 098, one launch, configurations in a shuffled order per cell) FreeToken ran in five settings per cell: hybrid with its calibrated fetch split (its default), hybrid with the fetch cap fixed at one and at two, hybrid with 8 CPU threads instead of one per physical core, and offload. Its best setting was the calibrated hybrid with 8 threads at three cells and offload at the other three (gpt-oss 25 and 40%, Qwen3 43.75%), at most 1.04 $\times$ its Table 2 setting; the fixed caps were never better than its own split by more than 0.3%. Ours over FreeToken’s best: 1.17, 1.17 and 1.09 on gpt-oss, 1.02, 1.04 and 0.98 on Qwen3, within 0.04 of host S’s ratios at every cell. With every expert on the CPU, stock llama.cpp decodes gpt-oss 1.13 $\times$ and Qwen3 0.99 $\times$ as fast as FreeToken’s CPU backend.

Other systems and outputs. Two newer llama.cpp-based systems, Pipelined Sharding [43] and leloch’s expert cache [25], run gpt-oss-120b at 1.28–1.45 $\times$ stock llama.cpp; ours ran 1.42–2.34 $\times$ the better of them (job 077). KTransformers [7] (kt-kernel 0.7.0, exact BF16 CPU experts, the same number of experts statically on the GPU) ran Qwen3 in BF16 at 0.98–1.01 $\times$ stock llama.cpp on an RTX 5090 with a Ryzen 9 9950X, and ours at 2.2–4.4 $\times$ it (job 078); both static placements read the same CPU expert bytes from the same DRAM. Its published speeds rely on AMX Xeons and INT4/INT8 CPU weights, which that host lacks and which change the model, so this is not its best setting. Teacher-forced on fixed text (1536 steps per model) against stock llama.cpp with every weight in the VRAM of an RTX PRO 6000 (job 090), our cache at 25% has a mean KL divergence over the full vocabulary of 0.0019 nats on gpt-oss and 0.0005 on Qwen3 (99.9th percentile 0.121 and 0.059, maximum 0.21 and 0.13), picks the same next token on 98.5% and 99.3% of steps, and changes the loss by +0.16% and +0.19%. Stock llama.cpp’s own --n-cpu-moe placement diverges from the same reference by as much (mean KL 0.0017 and 0.0008, agreement 98.8% and 99.3%): the differences are those of running experts on the CPU (the helpers quantise activations to Q8_0 where the GPU path uses Q8_1), not of the cache. Task accuracy was not measured.

Prompts and batches. The cache serves single-token steps; prompts and batches above one run llama.cpp’s stock graph with every expert in host memory, and pay for the cache’s resident experts and its one-thread CPU side. In llama-batched-bench at 25% on host S (job 089, both models) our prefill at 512 and 2,048 tokens runs 79–83% of

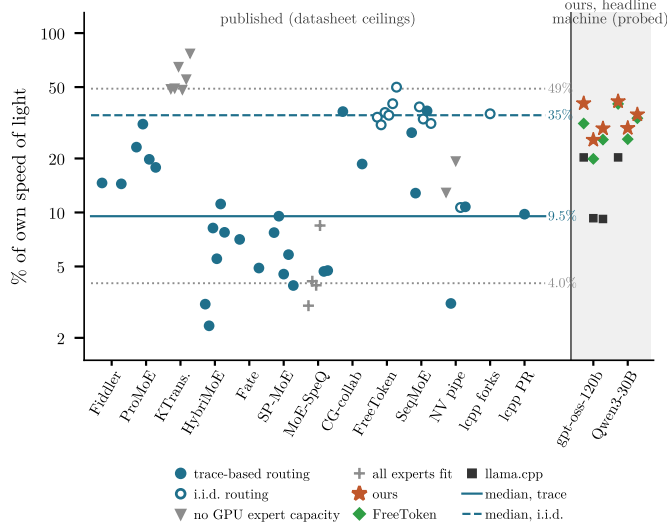


Figure 7: Published MoE-offloading measurements (52 rows from 13 sources) as a percentage of their own speed of light, computed from the paper’s hardware at datasheet rates and, where a routing trace of the model exists, the per-layer Belady optimum (filled); hollow points assume uniform routing, which overstates the optimum’s reads and so the percentage; the dotted lines are the class medians (no GPU expert capacity, 49.0%; every expert resident, 4.0%). Right: our cache, FreeToken and stock llama.cpp on host B against the probed per-layer limit of Table 2 (the pooled bound FreeToken’s and llama.cpp’s designs allow is 0–10% higher).

stock llama.cpp’s and our decode at 2 and 4 parallel sequences 77–79% of it (the tool feeds both engines the same random tokens).

H The audit’s rows

Table 14 lists the 52 published measurements of Section 3 with their bounds. The sources are Fiddler [22], ProMoE [39], KTransformers [7], HybriMoE [58], Fate [11], SP-MoE [8], MoE-SpeQ [44], the CPU–GPU collaborative inference of Huang et al. [17], FreeToken [51], SeqMoE [46], NVIDIA’s pipelined sharding [43], a llama.cpp fork [25] and a llama.cpp pull request [9]. Three caveats apply to every row: the ceilings are datasheet ones where ours are probed; the trace is the model’s own sampled text, sometimes of a sibling variant (an FP8 trace for a Q4 deployment); and systems whose cache is pooled across layers, whose placement pins whole layers, or which verify several tokens per pass are scored against the per-layer one-token bound, which is not theirs.

Table 14: The audit’s rows (Section 3): every published batch-1 measurement we scored, with the system’s stated hardware, its experts per layer resident on the GPU (C), the model’s experts and $\text{top-}k$, the published speed, the bound at datasheet-physical ceilings (GPU datasheet bandwidth, the top of the host-DRAM band, datasheet PCIe, no latencies; `scripts/audit.py`), how the optimum’s reads were obtained (*trace*: Belady’s MIN with bypass on an on-policy routing trace of the model; *uniform*: independent uniform routing, which overstates the reads and so the percentage; *no slots*: no GPU expert capacity, the CPU-only rate is the bound; *all fit*: every expert resident, nothing offloaded) and the percentage of the bound. Medians: trace 9.5% ($n=29$), uniform 34.8% (11), no slots 49.0% (8), all fit 4.0% (4), all rows 14.5% (52). Rows below 5% are more likely configuration mismatches with the bound’s assumptions (precision, dense layers on the CPU, end-to-end timing) than inefficiency, and are not adjudicated.

System	Model	GPU	C	E/k	tok/s	bound	reads	%
Fiddler [22]	Mixtral-8x7B	Quadro RTX 6000	1	8/2	2.5	17	trace	14.6
	Mixtral-8x7B	RTX 6000 Ada	3	8/2	8.3	57	trace	14.4
ProMoE [39]	DeepSeek-V2-Lite	RTX 4090	45	64/6	49.9	216	trace	23.1
	Qwen1.5-MoE-A2.7B	RTX 4090	46	60/4	67.2	216	trace	31.1
	Qwen2-57B-A14B	RTX 4090	42	64/8	26.9	136	trace	19.8

continued

System	Model	GPU	C	E/k	tok/s	bound	reads	%
KTransformers [7]	Mixtral-8x7B	RTX 4090	6	8/2	27.0	152	trace	17.8
	DeepSeek-V3-0324	A100 40GB	0	256/8	5.9	12	no slots	48.8
	DeepSeek-V2.5-1210	A100 40GB	0	160/6	11.9	24	no slots	49.2
	Qwen2-57B-A14B	A100 40GB	0	64/8	22.9	35	no slots	64.9
	DeepSeek-V3-0324	RTX 4080	0	256/8	16.1	33	no slots	48.3
HybriMoE [58]	DeepSeek-V2.5-1210	RTX 4080	0	160/6	17.6	32	no slots	55.3
	Qwen2-57B-A14B	RTX 4080	0	64/8	37.0	48	no slots	76.9
	DeepSeek-V2-Lite-Chat	RTX A6000	16	64/6	14.5	470	trace	3.1
	DeepSeek-V2-Lite-Chat	RTX A6000	48	64/6	15.3	655	trace	2.3
	Mixtral-8x7B-Instruct	RTX A6000	2	8/2	4.4	53	trace	8.2
Fate [11]	Mixtral-8x7B-Instruct	RTX A6000	6	8/2	7.3	133	trace	5.5
	Qwen2-57B-A14B-Instruct	RTX A6000	16	64/8	9.6	86	trace	11.1
	Qwen2-57B-A14B-Instruct	RTX A6000	48	64/8	9.0	116	trace	7.7
	Qwen1.5-MoE-A2.7B	RTX 3090	46	60/4	14.0	198	trace	7.1
	Qwen1.5-MoE-A2.7B	GTX 1080 Ti	15	60/4	3.7	76	trace	4.9
SP-MoE [8]	Mixtral-8x7B	RTX 3090	1	8/2	1.2	16	trace	7.7
	Mixtral-8x7B	RTX 4090	1	8/2	1.3	13	trace	9.5
	Mixtral-8x7B	A100 40GB	3	8/2	1.5	32	trace	4.5
	Phi-3.5-MoE	RTX 4090	3	16/2	3.3	57	trace	5.8
	DeepSeek-V2-Lite	A100 40GB	7	64/6	5.6	142	trace	3.9
MoE-SpeQ [44]	DeepSeek-V2-Lite	A100 40GB	64	64/6	10.1	335	all fit	3.0
	DeepSeek-V2-Lite	A100 40GB	64	64/6	13.8	335	all fit	4.1
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	13.2	335	all fit	3.9
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	28.3	335	all fit	8.5
	Phi-3.5-MoE	A100 40GB	7	16/2	6.0	129	trace	4.7
CPU-GPU collab. [17]	Phi-3.5-MoE	A100 40GB	7	16/2	6.1	129	trace	4.7
	Mixtral-8x7B	RTX 4090	1	8/2	4.8	13	trace	36.5
	Phi-3.5-MoE	RTX 4090	3	16/2	10.4	56	trace	18.6
FreeToken [51]	Qwen3.6-35B-A3B	RTX 5090	94	256/8	77.1	227	uniform	34.0
	DeepSeek-V4-Flash	RTX 5090	28	256/6	24.9	81	uniform	30.8
	Qwen3.6-35B-A3B	RTX 5090	104	256/8	73.8	205	uniform	36.1
	Qwen3.6-35B-A3B	RTX 4090	72	256/8	42.9	123	uniform	34.8
	Qwen3.6-35B-A3B	RTX 4060 Laptop	60	256/8	39.3	97	uniform	40.4
SeqMoE [46]	GLM-5.2	RTX PRO 6000 Blackwell	36	256/8	14.9	30	uniform	49.9
	Qwen3-30B-A3B-FP8	RTX 4090	57	128/8	104.1	374	trace	27.9
	Qwen3-30B-A3B-FP8	RTX 4090	19	128/8	47.9	374	trace	12.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	102	256/8	118.3	305	uniform	38.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	38	256/8	70.2	212	uniform	33.2
Pipelined sharding [43]	GPT-OSS-120B-MXFP4	RTX PRO 6000 Blackwell	57	128/4	139.2	378	trace	36.8
	DeepSeek-V4-Flash	RTX PRO 6000 Blackwell	115	256/6	57.4	183	uniform	31.3
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	44	128/8	32.1	1032	trace	3.1
	Qwen3-235B-A22B-Instruct-2507	RTX 5090	0	128/8	7.7	40	no slots	19.3
llama.cpp fork (leloch) [25]	Qwen3-235B-A22B-Instruct-2507	RTX 5090	42	128/8	11.5	108	uniform	10.7
	Qwen3-30B-A3B-Instruct-2507	RTX 5070 Ti	107	128/8	54.9	511	trace	10.8
	Laguna S 2.1	RTX 5090	66	256/10	31.3	88	uniform	35.5
	Qwen3-30B-A3B Q4_K_M derivative	Radeon RX 7600 8GB	32	128/8	16.5	169	trace	9.8

I Artifact description

- **System.** A patch to llama.cpp commit 4da6337 (the expert cache, FETCH, the mailbox and the llama-ec-bench instrumentation: teacher-forced loss, logit dumps, routing lookahead records); built with CUDA 13.0 for compute capability 12.0.
- **Jobs.** One shell script per job; each installs FreeToken at commit 0d652e7, downloads the models from Hugging Face, runs the bandwidth probes, writes any on-machine prediction, runs the client, and stores raw rows (one JSON line per problem), server logs and engine counters. A small runner rents a Vast instance, streams a heartbeat, fetches the results and only then destroys the instance.
- **Analysis.** scripts/*_stats.py compute every interval from the raw rows; scripts/perlayer_model.py, speed_limit_v2.py, shapley_gap.py, policy_study.py, foresight_exact.py, batchk_trace.py, fig_audit_sol.py and scorecard.py produce the model comparison, the limit, the accounting, the trace studies, the audit figure and the scorecard; scripts/wsg_tables.py and wsg_numbers2.py generate every table, figure and number in this paper from their outputs. The audit's 52 published rows and the physical speed-of-light model they are scored against are data/published_measurements.json, data/audit/ and scripts/audit.py.

- **Cost.** The 66 rentals (39 distinct machines) of jobs 058–098 cost 55.9 US dollars at their hourly GPU prices, by the ledger in the repository; storage and download charges, which the ledger omits, add to this.